Heurísticas y búsqueda local

Búsqueda local, heurísticas, aproximación, metaheurísticas (GRASP) y A*

Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

2do Cuatrimestre de 2026

Agenda de hoy

1. ¿Y si el problema es difícil? Qué se puede ceder.
2. Búsqueda local: vecindades, óptimos locales; *n* reinas y TSP con 2-opt.
3. Heurísticas: qué son, cómo se construyen, cómo se evalúan.
4. Algoritmos de aproximación: garantías demostradas (vertex cover, TSP métrico).
5. Metaheurísticas: escapar de óptimos locales; GRASP.
6. A*: una heurística que *no* sacrifica optimalidad.
7. Glosario, preguntas de parcial, tips.

Esta clase no tiene guía de ejercicios; sí puede haber **preguntas teóricas** sobre estos conceptos en el parcial.
Referencias: Kleinberg–Tardos caps. 11 y 12; Cormen et al. cap. 35; Russell–Norvig caps. 3, 4 y 6.

Un viejo conocido

Problema del viajante de comercio (TSP)

Dado K_n con pesos $d(u, v) \geq 0$, encontrar un ciclo que pase por todos los vértices exactamente una vez (un *tour*) de costo mínimo.

Lo que ya sabemos hacer con él:

Un viejo conocido

Problema del viajante de comercio (TSP)

Dado K_n con pesos $d(u, v) \geq 0$, encontrar un ciclo que pase por todos los vértices exactamente una vez (un *tour*) de costo mínimo.

Lo que ya sabemos hacer con él:

- **Fuerza bruta:** $\frac{(n-1)!}{2}$ tours.

Un viejo conocido

Problema del viajante de comercio (TSP)

Dado K_n con pesos $d(u, v) \geq 0$, encontrar un ciclo que pase por todos los vértices exactamente una vez (un *tour*) de costo mínimo.

Lo que ya sabemos hacer con él:

- **Fuerza bruta:** $\frac{(n-1)!}{2}$ tours.
- **Backtracking con podas** (clase 5): mejor en la práctica, mismo peor caso.

Un viejo conocido

Problema del viajante de comercio (TSP)

Dado K_n con pesos $d(u, v) \geq 0$, encontrar un ciclo que pase por todos los vértices exactamente una vez (un *tour*) de costo mínimo.

Lo que ya sabemos hacer con él:

- **Fuerza bruta:** $\frac{(n-1)!}{2}$ tours.
- **Backtracking con podas** (clase 5): mejor en la práctica, mismo peor caso.
- **Programación dinámica** (Held–Karp: la idea de la clase 8 aplicada al TSP): $O(n^2\, 2^n)$.

Un viejo conocido

Problema del viajante de comercio (TSP)

Dado K_n con pesos $d(u, v) \geq 0$, encontrar un ciclo que pase por todos los vértices exactamente una vez (un *tour*) de costo mínimo.

Lo que ya sabemos hacer con él:

- **Fuerza bruta:** $\frac{(n-1)!}{2}$ tours.
- **Backtracking con podas** (clase 5): mejor en la práctica, mismo peor caso.
- **Programación dinámica** (Held–Karp: la idea de la clase 8 aplicada al TSP): $O(n^2\, 2^n)$.

n	10	20	40	100
Fuerza bruta	$1{,}8 \times 10^5$	6×10^{16}	1×10^{46}	5×10^{155}
Held–Karp	10^5	4×10^8	2×10^{15}	1×10^{34}

Mientras tanto, en el mundo real

- El TSP es **NP-difícil**[1]: nadie conoce un algoritmo exacto polinomial, y probablemente no exista.

[1] Informalmente: si alguien encontrara un algoritmo polinomial para el TSP, habría algoritmos polinomiales para todos los problemas de NP. Nadie lo encontró, y la conjetura estándar es que no existe ($P \neq NP$).

Mientras tanto, en el mundo real

- El TSP es **NP-difícil**[1]: nadie conoce un algoritmo exacto polinomial, y probablemente no exista.
- La instancia `pla85900` (85 900 ciudades) se resolvió a optimalidad en 2006 (Applegate, Bixby, Chvátal y Cook, *Concorde*), con un costo del orden de **136 años de CPU**.

[1] Informalmente: si alguien encontrara un algoritmo polinomial para el TSP, habría algoritmos polinomiales para todos los problemas de NP. Nadie lo encontró, y la conjetura estándar es que no existe (P $\neq$ NP).

Mientras tanto, en el mundo real

- El TSP es **NP-difícil**[1]: nadie conoce un algoritmo exacto polinomial, y probablemente no exista.
- La instancia `pla85900` (85 900 ciudades) se resolvió a optimalidad en 2006 (Applegate, Bixby, Chvátal y Cook, *Concorde*), con un costo del orden de **136 años de CPU**.
- La misma instancia: una buena heurística (LKH, Helsgaun) llega a menos de 0,1 % del óptimo en **horas**, en una sola máquina.

[1] Informalmente: si alguien encontrara un algoritmo polinomial para el TSP, habría algoritmos polinomiales para todos los problemas de NP. Nadie lo encontró, y la conjetura estándar es que no existe (P $\neq$ NP).

Mientras tanto, en el mundo real

- El TSP es **NP-difícil**[1]: nadie conoce un algoritmo exacto polinomial, y probablemente no exista.
- La instancia `pla85900` (85 900 ciudades) se resolvió a optimalidad en 2006 (Applegate, Bixby, Chvátal y Cook, *Concorde*), con un costo del orden de **136 años de CPU**.
- La misma instancia: una buena heurística (LKH, Helsgaun) llega a menos de 0,1 % del óptimo en **horas**, en una sola máquina.
- Si mañana a las 6 hay que decidir el recorrido de 300 camiones, ¿cuál preferís?

[1] Informalmente: si alguien encontrara un algoritmo polinomial para el TSP, habría algoritmos polinomiales para todos los problemas de NP. Nadie lo encontró, y la conjetura estándar es que no existe (P $\neq$ NP).

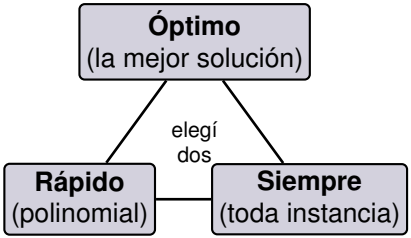
Mientras tanto, en el mundo real

- El TSP es **NP-difícil**[1]: nadie conoce un algoritmo exacto polinomial, y probablemente no exista.
- La instancia `pla85900` (85 900 ciudades) se resolvió a optimalidad en 2006 (Applegate, Bixby, Chvátal y Cook, *Concorde*), con un costo del orden de **136 años de CPU**.
- La misma instancia: una buena heurística (LKH, Helsgaun) llega a menos de 0,1 % del óptimo en **horas**, en una sola máquina.
- Si mañana a las 6 hay que decidir el recorrido de 300 camiones, ¿cuál preferís?

Hoy: qué hacemos cuando el problema es difícil o la instancia es grande y los algoritmos exactos no llegan. Y, sobre todo, **qué estamos cediendo** en cada caso.

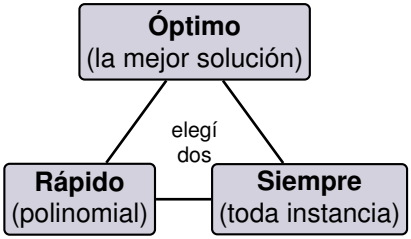
[1] Informalmente: si alguien encontrara un algoritmo polinomial para el TSP, habría algoritmos polinomiales para todos los problemas de NP. Nadie lo encontró, y la conjetura estándar es que no existe ($P \neq NP$).

Tres cosas que queremos y no podemos tener juntas



Para un problema NP-difícil (salvo que P $=$ NP), hay que **ceder** una:

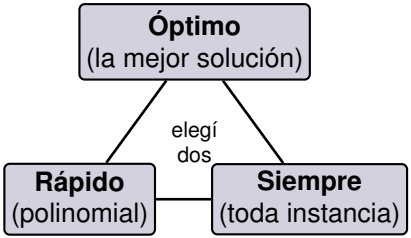
Tres cosas que queremos y no podemos tener juntas



Para un problema NP-difícil (salvo que P $=$ NP), hay que **ceder** una:

- **Rapidez:** backtracking, branch and bound, DP exponencial. *Ya lo hicimos.*

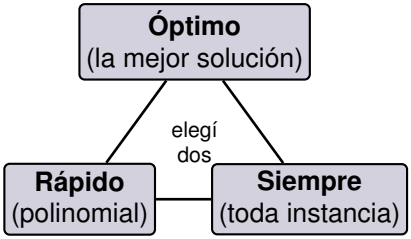
Tres cosas que queremos y no podemos tener juntas



Para un problema NP-difícil (salvo que P $=$ NP), hay que **ceder** una:

- **Rapidez:** backtracking, branch and bound, DP exponencial. *Ya lo hicimos.*
- **Generalidad:** casos especiales (árboles, DAGs, bipartitos, instancias chicas). *También.*

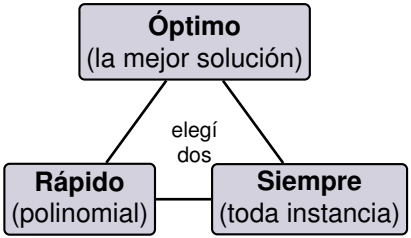
Tres cosas que queremos y no podemos tener juntas



Para un problema NP-difícil (salvo que P $=$ NP), hay que **ceder** una:

- **Rapidez:** backtracking, branch and bound, DP exponencial. *Ya lo hicimos.*
- **Generalidad:** casos especiales (árboles, DAGs, bipartitos, instancias chicas). *También.*
- **Optimalidad:** conformarse con una solución *buena*. Hoy. Sin garantía (*heurísticas, búsqueda local, metaheurísticas*) o con garantía demostrada (*aproximación*).

Tres cosas que queremos y no podemos tener juntas



Para un problema NP-difícil (salvo que P $=$ NP), hay que **ceder** una:

- **Rapidez:** backtracking, branch and bound, DP exponencial. *Ya lo hicimos.*
- **Generalidad:** casos especiales (árboles, DAGs, bipartitos, instancias chicas). *También.*
- **Optimalidad:** conformarse con una solución *buena*. Hoy. Sin garantía (*heurísticas, búsqueda local, metaheurísticas*) o con garantía demostrada (*aproximación*).

Y una idea transversal, al final: **usar información** del problema para explorar menos *sin ceder nada* (A*).

El glosario de hoy (a completar)

Término	¿Qué es? ¿Qué garantiza?
Búsqueda local	
Heurística	
Algoritmo de aproximación	
Metaheurística (GRASP)	
A*	

El glosario de hoy (a completar)

Término	¿Qué es? ¿Qué garantiza?
Búsqueda local	
Heurística	
Algoritmo de aproximación	
Metaheurística (GRASP)	
A*	

Al final volvemos a esta tabla. Cada definición entra en una línea, y cada término tiene un *ejemplo canónico* que hay que saber.

Qué tienen que saber al terminar

1. **Definir** con precisión los cinco términos, con un ejemplo de cada uno.

Qué tienen que saber al terminar

1. **Definir** con precisión los cinco términos, con un ejemplo de cada uno.
2. **Distinguir** los pares que se confunden: heurística vs. aproximación; heurística (algoritmo) vs. heurística (función h); admisible vs. consistente; óptimo local vs. global.

Qué tienen que saber al terminar

1. **Definir** con precisión los cinco términos, con un ejemplo de cada uno.
2. **Distinguir** los pares que se confunden: heurística vs. aproximación; heurística (algoritmo) vs. heurística (función h); admisible vs. consistente; óptimo local vs. global.
3. **Enunciar y demostrar** dos aproximaciones: vertex cover por matching maximal y TSP métrico por árbol generador mínimo.

Qué tienen que saber al terminar

1. **Definir** con precisión los cinco términos, con un ejemplo de cada uno.
2. **Distinguir** los pares que se confunden: heurística vs. aproximación; heurística (algoritmo) vs. heurística (función h); admisible vs. consistente; óptimo local vs. global.
3. **Enunciar y demostrar** dos aproximaciones: vertex cover por matching maximal y TSP métrico por árbol generador mínimo.
4. **Explicar** GRASP: sus dos fases, el rol de α, cómo se evalúa.

Qué tienen que saber al terminar

1. **Definir** con precisión los cinco términos, con un ejemplo de cada uno.
2. **Distinguir** los pares que se confunden: heurística vs. aproximación; heurística (algoritmo) vs. heurística (función h); admisible vs. consistente; óptimo local vs. global.
3. **Enunciar y demostrar** dos aproximaciones: vertex cover por matching maximal y TSP métrico por árbol generador mínimo.
4. **Explicar** GRASP: sus dos fases, el rol de α, cómo se evalúa.
5. **Explicar** A*: qué es $f = g + h$, qué pide de h y qué garantiza.

Qué tienen que saber al terminar

1. **Definir** con precisión los cinco términos, con un ejemplo de cada uno.
2. **Distinguir** los pares que se confunden: heurística vs. aproximación; heurística (algoritmo) vs. heurística (función h); admisible vs. consistente; óptimo local vs. global.
3. **Enunciar y demostrar** dos aproximaciones: vertex cover por matching maximal y TSP métrico por árbol generador mínimo.
4. **Explicar** GRASP: sus dos fases, el rol de α, cómo se evalúa.
5. **Explicar** A*: qué es $f = g + h$, qué pide de h y qué garantiza.

No hay práctica de esto. Pero aparece en el TP de heurísticas, en el parcial y en cualquier trabajo donde haya que optimizar algo de verdad.

Problemas de optimización combinatoria

Marco

Una **instancia** I determina un conjunto **finito** de soluciones factibles $\text{Sols}_{\text{válidas}}$ y una función de costo $c : \text{Sols}_{\text{válidas}} \to \mathbb{R}$. Queremos $\text{OPT}(I) = \text{mín}\{c(s) : s \in \text{Sols}_{\text{válidas}}\}$ (o máx).

Problemas de optimización combinatoria

Marco

Una **instancia** I determina un conjunto **finito** de soluciones factibles $\text{Sols}_{\text{válidas}}$ y una función de costo $c : \text{Sols}_{\text{válidas}} \to \mathbb{R}$. Queremos $\text{OPT}(I) = \text{mín}\{c(s) : s \in \text{Sols}_{\text{válidas}}\}$ (o máx).

- **TSP:** $\text{Sols}_{\text{válidas}}$ = tours; c = suma de distancias.

Problemas de optimización combinatoria

Marco

Una **instancia** I determina un conjunto **finito** de soluciones factibles $\text{Sols}_{\text{válidas}}$ y una función de costo $c : \text{Sols}_{\text{válidas}} \to \mathbb{R}$. Queremos $\text{OPT}(I) = \text{mín}\{c(s) : s \in \text{Sols}_{\text{válidas}}\}$ (o máx).

- **TSP:** $\text{Sols}_{\text{válidas}}$ = tours; c = suma de distancias.
- n **reinas como optimización:** $\text{Sols}_{\text{válidas}} = \{0, \ldots, n-1\}^n$ (fila de la reina de cada columna); $c(s)$ = pares de reinas que se atacan. Solución del problema original $\Leftrightarrow$ $c = 0$.

Problemas de optimización combinatoria

Marco

Una **instancia** I determina un conjunto **finito** de soluciones factibles $\text{Sols}_{\text{válidas}}$ y una función de costo $c : \text{Sols}_{\text{válidas}} \to \mathbb{R}$. Queremos $\text{OPT}(I) = \text{mín}\{c(s) : s \in \text{Sols}_{\text{válidas}}\}$ (o máx).

- **TSP:** $\text{Sols}_{\text{válidas}}$ = tours; c = suma de distancias.
- n **reinas como optimización:** $\text{Sols}_{\text{válidas}} = \{0, \ldots, n-1\}^n$ (fila de la reina de cada columna); $c(s)$ = pares de reinas que se atacan. Solución del problema original $\Leftrightarrow$ $c = 0$.
- **Vertex cover:** $\text{Sols}_{\text{válidas}}$ = conjuntos de vértices que tocan toda arista; c = tamaño.

Problemas de optimización combinatoria

Marco

Una **instancia** I determina un conjunto **finito** de soluciones factibles $\text{Sols}_{\text{válidas}}$ y una función de costo $c : \text{Sols}_{\text{válidas}} \to \mathbb{R}$. Queremos $\text{OPT}(I) = \text{mín}\{c(s) : s \in \text{Sols}_{\text{válidas}}\}$ (o máx).

- **TSP:** $\text{Sols}_{\text{válidas}}$ = tours; c = suma de distancias.
- n **reinas como optimización:** $\text{Sols}_{\text{válidas}} = \{0, \ldots, n-1\}^n$ (fila de la reina de cada columna); $c(s)$ = pares de reinas que se atacan. Solución del problema original $\Leftrightarrow$ $c = 0$.
- **Vertex cover:** $\text{Sols}_{\text{válidas}}$ = conjuntos de vértices que tocan toda arista; c = tamaño.
- **Coloreo:** $\text{Sols}_{\text{válidas}}$ = coloreos propios; c = cantidad de colores.

Problemas de optimización combinatoria

Marco

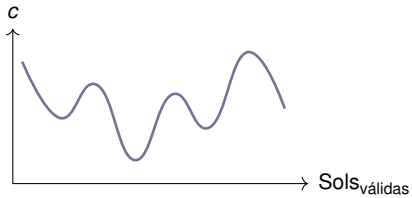
Una **instancia** I determina un conjunto **finito** de soluciones factibles $\text{Sols}_{\text{válidas}}$ y una función de costo $c : \text{Sols}_{\text{válidas}} \to \mathbb{R}$. Queremos $\text{OPT}(I) = \text{mín}\{c(s) : s \in \text{Sols}_{\text{válidas}}\}$ (o máx).

- **TSP:** $\text{Sols}_{\text{válidas}}$ = tours; c = suma de distancias.
- n **reinas como optimización:** $\text{Sols}_{\text{válidas}} = \{0, \ldots, n-1\}^n$ (fila de la reina de cada columna); $c(s)$ = pares de reinas que se atacan. Solución del problema original $\Leftrightarrow$ $c = 0$.
- **Vertex cover:** $\text{Sols}_{\text{válidas}}$ = conjuntos de vértices que tocan toda arista; c = tamaño.
- **Coloreo:** $\text{Sols}_{\text{válidas}}$ = coloreos propios; c = cantidad de colores.

Mismo marco que en la clase 5 (candidatas, válidas, valor). La diferencia de hoy: **no vamos a recorrer** $\text{Sols}_{\text{válidas}}$; vamos a *caminar* por él.

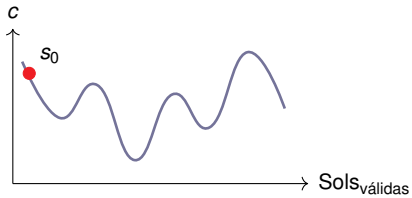
La idea: mejorar de a poco

- Arrancar en *cualquier* solución factible s.



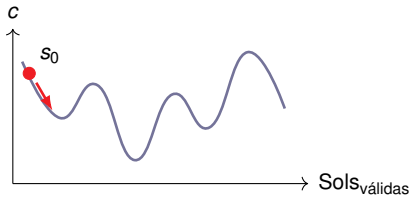
La idea: mejorar de a poco

- Arrancar en *cualquier* solución factible s.
- Mirar sus **vecinas**: las soluciones «cercanas».



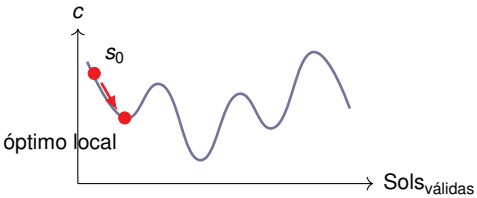
La idea: mejorar de a poco

- Arrancar en *cualquier* solución factible s.
- Mirar sus **vecinas**: las soluciones «cercanas».
- Si alguna es mejor, moverse. Repetir.



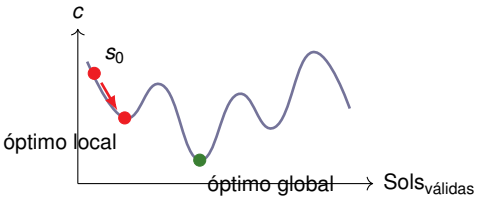
La idea: mejorar de a poco

- Arrancar en *cualquier* solución factible s.
- Mirar sus **vecinas**: las soluciones «cercanas».
- Si alguna es mejor, moverse. Repetir.
- Parar cuando ninguna mejora.



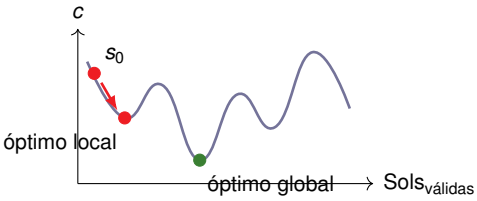
La idea: mejorar de a poco

- Arrancar en *cualquier* solución factible *s*.
- Mirar sus **vecinas**: las soluciones «cercanas».
- Si alguna es mejor, moverse. Repetir.
- Parar cuando ninguna mejora.
- Bajar una montaña con niebla.



La idea: mejorar de a poco

- Arrancar en *cualquier* solución factible s.
- Mirar sus **vecinas**: las soluciones «cercanas».
- Si alguna es mejor, moverse. Repetir.
- Parar cuando ninguna mejora.
- Bajar una montaña con niebla.



Definiciones

Vecindad: una función $\mathcal{N} : \text{Sols}_{\text{válidas}} \to 2^{\text{Sols}_{\text{válidas}}}$; $\mathcal{N}(s)$ son las *vecinas* de s. **Óptimo local** (para $\mathcal{N}$): una s con $c(s) \leq c(s')$ para toda $s' \in \mathcal{N}(s)$. **Óptimo global:** un mínimo de c sobre todo $\text{Sols}_{\text{válidas}}$.

Búsqueda local

function busquedaLocal(s_0)
    $s \leftarrow s_0$;
    while *existe* $s' \in \mathcal{N}(s)$ *con* $c(s') < c(s)$ **do**
        $s \leftarrow s'$; `// first improvement / best improvement`
    return s;

Búsqueda local

function busquedaLocal(s_0)
    $s \leftarrow s_0$;
    while *existe* $s' \in \mathcal{N}(s)$ *con* $c(s') < c(s)$ **do**
        $s \leftarrow s'$; `// first improvement / best improvement`
    return s;

- **Termina:** $c(s)$ decrece estrictamente y toma finitos valores. $\longrightarrow$ Pizarrón.

Búsqueda local

function busquedaLocal(s_0)
    $s \leftarrow s_0$;
    while *existe* $s' \in \mathcal{N}(s)$ *con* $c(s') < c(s)$ **do**
        $s \leftarrow s'$; `// first improvement / best improvement`
    return s;

- **Termina:** $c(s)$ decrece estrictamente y toma finitos valores. $\longrightarrow$ Pizarrón.
- **Devuelve un óptimo local.** Es lo único que garantiza. No dice nada sobre OPT.

Búsqueda local

function busquedaLocal(s_0)
  $s \leftarrow s_0$;
  while *existe* $s' \in \mathcal{N}(s)$ *con* $c(s') < c(s)$ **do**
    $s \leftarrow s'$; `// first improvement / best improvement`
  return s;

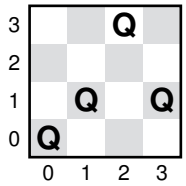
- **Termina:** $c(s)$ decrece estrictamente y toma finitos valores. $\longrightarrow$ Pizarrón.
- **Devuelve un óptimo local.** Es lo único que garantiza. No dice nada sobre OPT.
- **Costo:** iteraciones $\times$ (recorrer $\mathcal{N}(s)$ y evaluar cada vecina). Lo segundo lo controlamos; lo primero, en general, no.

Búsqueda local

function busquedaLocal(s_0)
  $s \leftarrow s_0$;
  while *existe* $s' \in \mathcal{N}(s)$ *con* $c(s') < c(s)$ **do**
    $s \leftarrow s'$; `// first improvement / best improvement`
  return s;

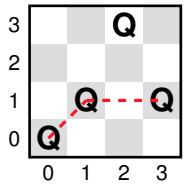
- **Termina:** $c(s)$ decrece estrictamente y toma finitos valores. $\longrightarrow$ Pizarrón.
- **Devuelve un óptimo local.** Es lo único que garantiza. No dice nada sobre OPT.
- **Costo:** iteraciones $\times$ (recorrer $\mathcal{N}(s)$ y evaluar cada vecina). Lo segundo lo controlamos; lo primero, en general, no.
- **Decisiones de diseño:** representación, vecindad, evaluación (idealmente *incremental*: $c(s') - c(s)$ sin recalcular todo), regla de selección.

n reinas como búsqueda local



- **Representación:** $s \in \{0, \ldots, n-1\}^n$; s_j = fila de la reina de la columna j. Toda s es factible.

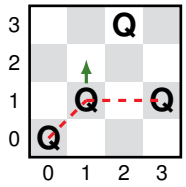
n reinas como búsqueda local



$s = (0, 1, 3, 1)$, $c(s) = 2$.

- **Representación:** $s \in \{0, \ldots, n-1\}^n$; s_j = fila de la reina de la columna j. Toda s es factible.
- **Costo:** pares (i, j) con $s_i = s_j$ o $|s_i - s_j| = |i - j|$.

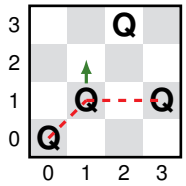
n reinas como búsqueda local



$s = (0, 1, 3, 1)$, $c(s) = 2$. La vecina $(0, 2, 3, 1)$ tiene costo 1.

- **Representación:** $s \in \{0, \ldots, n-1\}^n$; s_j = fila de la reina de la columna j. Toda s es factible.
- **Costo:** pares (i, j) con $s_i = s_j$ o $|s_i - s_j| = |i - j|$.
- **Vecindad:** mover *una* reina dentro de su columna. $|\mathcal{N}(s)| = n(n-1)$.

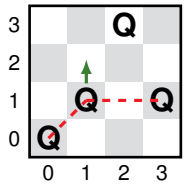
n reinas como búsqueda local



$s = (0, 1, 3, 1)$, $c(s) = 2$. La vecina $(0, 2, 3, 1)$ tiene costo 1.

- **Representación:** $s \in \{0, \ldots, n-1\}^n$; s_j = fila de la reina de la columna j. Toda s es factible.
- **Costo:** pares (i, j) con $s_i = s_j$ o $|s_i - s_j| = |i - j|$.
- **Vecindad:** mover *una* reina dentro de su columna. $|\mathcal{N}(s)| = n(n-1)$.
- **Evaluación incremental:** mover la reina j cambia sólo los pares que la involucran: $O(n)$ por vecina.

n reinas como búsqueda local



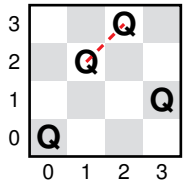
$s = (0, 1, 3, 1)$, $c(s) = 2$. La vecina $(0, 2, 3, 1)$ tiene costo 1.

- **Representación:** $s \in \{0, \ldots, n-1\}^n$; s_j = fila de la reina de la columna j. Toda s es factible.
- **Costo:** pares (i, j) con $s_i = s_j$ o $|s_i - s_j| = |i - j|$.
- **Vecindad:** mover *una* reina dentro de su columna. $|\mathcal{N}(s)| = n(n-1)$.
- **Evaluación incremental:** mover la reina j cambia sólo los pares que la involucran: $O(n)$ por vecina.

Min-conflicts (Minton et al., 1992)

Elegir al azar una columna en conflicto; mover su reina a la fila que *minimiza* los conflictos. Con inicio greedy resuelve $n = 10^6$ en **decenas de movimientos** en promedio. Backtracking, para ese n, es impensable.

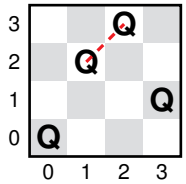
Óptimo local $\neq$ óptimo global



$s = (0, 2, 3, 1)$, $c(s) = 1$

- Un solo conflicto (columnas 1 y 2, en diagonal).

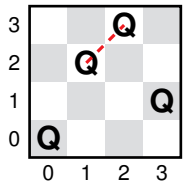
Óptimo local $\neq$ óptimo global



$s = (0, 2, 3, 1)$, $c(s) = 1$

- Un solo conflicto (columnas 1 y 2, en diagonal).
- Las 12 vecinas tienen costo ≥ 1: una da 1 (una **meseta**), las otras once empeoran. $\longrightarrow$ Tabla en el pizarrón.

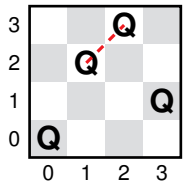
Óptimo local $\neq$ óptimo global



$s = (0, 2, 3, 1)$, $c(s) = 1$

- Un solo conflicto (columnas 1 y 2, en diagonal).
- Las 12 vecinas tienen costo ≥ 1: una da 1 (una **meseta**), las otras once empeoran. $\longrightarrow$ Tabla en el pizarrón.
- Es un **óptimo local** que no es solución. La búsqueda local se detiene a un paso de la respuesta, y no lo sabe.

Óptimo local $\neq$ óptimo global



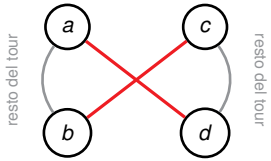
$s = (0, 2, 3, 1)$, $c(s) = 1$

- Un solo conflicto (columnas 1 y 2, en diagonal).
- Las 12 vecinas tienen costo ≥ 1: una da 1 (una **meseta**), las otras once empeoran. $\longrightarrow$ Tabla en el pizarrón.
- Es un **óptimo local** que no es solución. La búsqueda local se detiene a un paso de la respuesta, y no lo sabe.

¿Qué hacemos?

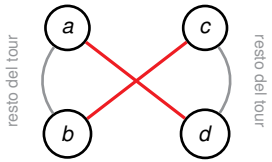
Reiniciar desde otra solución al azar; **aceptar** movimientos que no mejoran o que empeoran (metaheurísticas, sección 5); **agrandar** la vecindad (mover dos reinas): más caro, menos óptimos locales.

TSP: la vecindad 2-opt



- **Representación:** una permutación $(t_1, \ldots, t_n)$, cíclica.

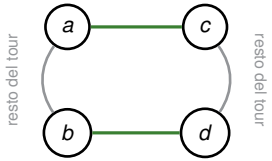
TSP: la vecindad 2-opt



Tour $\ldots a\, d \ldots c\, b \ldots$: quitamos $\{a, d\}$ y $\{c, b\}$.

- **Representación:** una permutación $(t_1, \ldots, t_n)$, cíclica.
- **Vecindad 2-opt:** quitar dos aristas no adyacentes y reconectar de la única otra forma posible. $|\mathcal{N}(t)| = \frac{n(n-3)}{2}$.

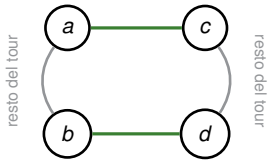
TSP: la vecindad 2-opt



Tour $\ldots a\, d \ldots c\, b \ldots$: quitamos $\{a, d\}$ y $\{c, b\}$. Reconectamos con $\{a, c\}$ y $\{b, d\}$: el segmento $d \ldots c$ queda invertido.

- **Representación:** una permutación $(t_1, \ldots, t_n)$, cíclica.
- **Vecindad 2-opt:** quitar dos aristas no adyacentes y reconectar de la única otra forma posible. $|\mathcal{N}(t)| = \frac{n(n-3)}{2}$.
- **Evaluación incremental en $O(1)$:** $\Delta = d(a, c) + d(b, d) - d(a, d) - d(b, c)$.

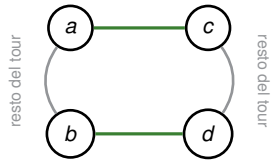
TSP: la vecindad 2-opt



Tour $\ldots a\, d \ldots c\, b \ldots$: quitamos $\{a, d\}$ y $\{c, b\}$. Reconectamos con $\{a, c\}$ y $\{b, d\}$: el segmento $d \ldots c$ queda invertido.

- **Representación:** una permutación $(t_1, \ldots, t_n)$, cíclica.
- **Vecindad 2-opt:** quitar dos aristas no adyacentes y reconectar de la única otra forma posible. $|\mathcal{N}(t)| = \frac{n(n-3)}{2}$.
- **Evaluación incremental en $O(1)$:** $\Delta = d(a, c) + d(b, d) - d(a, d) - d(b, c)$.
- Una pasada por la vecindad: $\Theta(n^2)$. Aplicar el movimiento: $O(n)$.

TSP: la vecindad 2-opt

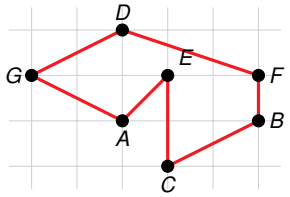


Tour $\ldots a\, d \ldots c\, b \ldots$: quitamos $\{a, d\}$ y $\{c, b\}$. Reconectamos con $\{a, c\}$ y $\{b, d\}$: el segmento $d \ldots c$ queda invertido.

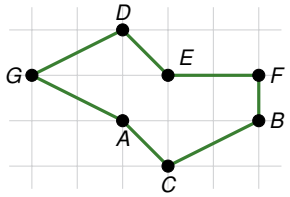
- **Representación:** una permutación $(t_1, \ldots, t_n)$, cíclica.
- **Vecindad 2-opt:** quitar dos aristas no adyacentes y reconectar de la única otra forma posible. $|\mathcal{N}(t)| = \frac{n(n-3)}{2}$.
- **Evaluación incremental en $O(1)$:** $\Delta = d(a, c) + d(b, d) - d(a, d) - d(b, c)$.
- Una pasada por la vecindad: $\Theta(n^2)$. Aplicar el movimiento: $O(n)$.

Observación (en el plano): si dos aristas del tour se cruzan, el 2-opt que las descruza *mejora estrictamente* (desigualdad triangular). Luego un óptimo local 2-opt **no tiene cruces**. La recíproca es falsa:

Un óptimo local 2-opt que no es óptimo

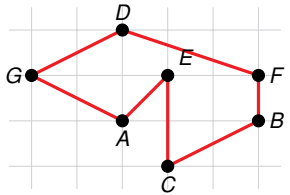


Óptimo local 2-opt: *AECBFDG*, costo 14,28.

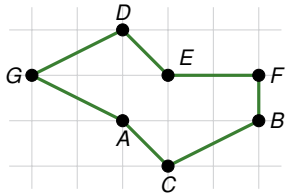


Óptimo global: *ACBFEDG*, costo 12,54.

Un óptimo local 2-opt que no es óptimo



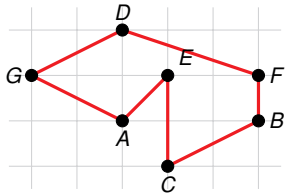
Óptimo local 2-opt: *AECBFDG*, costo 14,28.



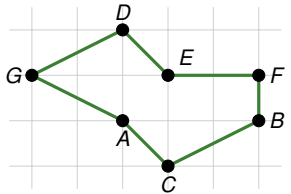
Óptimo global: *ACBFEDG*, costo 12,54.

- Siete puntos con coordenadas enteras, distancias euclídeas. El tour de la izquierda no tiene cruces y *ninguno* de sus 14 movimientos 2-opt lo mejora (verificado por fuerza bruta). Está 14 % por encima del óptimo.

Un óptimo local 2-opt que no es óptimo



Óptimo local 2-opt: *AECBFDG*, costo 14,28.



Óptimo global: *ACBFEDG*, costo 12,54.

- Siete puntos con coordenadas enteras, distancias euclídeas. El tour de la izquierda no tiene cruces y *ninguno* de sus 14 movimientos 2-opt lo mejora (verificado por fuerza bruta). Está 14 % por encima del óptimo.
- Para salir hace falta mover *tres* aristas a la vez (3-opt), o aceptar empeorar un rato.

¿Cuánto cuesta y cuánto vale 2-opt?

- **Por iteración:** $\Theta(n^2)$ vecinas, $O(1)$ cada una. Con «listas de vecinos» (los k más cercanos de cada ciudad), $O(kn)$ en la práctica.

¿Cuánto cuesta y cuánto vale 2-opt?

- **Por iteración:** $\Theta(n^2)$ vecinas, $O(1)$ cada una. Con «listas de vecinos» (los k más cercanos de cada ciudad), $O(kn)$ en la práctica.
- **Iteraciones:** en el peor caso, **exponenciales** en n, incluso en el plano (Englert, Röglin y Vöcking, 2014). En la práctica, pocas pasadas.

¿Cuánto cuesta y cuánto vale 2-opt?

- **Por iteración:** $\Theta(n^2)$ vecinas, $O(1)$ cada una. Con «listas de vecinos» (los k más cercanos de cada ciudad), $O(kn)$ en la práctica.
- **Iteraciones:** en el peor caso, **exponenciales** en n, incluso en el plano (Englert, Röglin y Vöcking, 2014). En la práctica, pocas pasadas.
- **Calidad, en general:** sin desigualdad triangular hay instancias donde los óptimos locales de k-opt son **arbitrariamente malos**, para todo k fijo (Papadimitriou y Steiglitz, 1978).

¿Cuánto cuesta y cuánto vale 2-opt?

- **Por iteración:** $\Theta(n^2)$ vecinas, $O(1)$ cada una. Con «listas de vecinos» (los k más cercanos de cada ciudad), $O(kn)$ en la práctica.
- **Iteraciones:** en el peor caso, **exponenciales** en n, incluso en el plano (Englert, Röglin y Vöcking, 2014). En la práctica, pocas pasadas.
- **Calidad, en general:** sin desigualdad triangular hay instancias donde los óptimos locales de k-opt son **arbitrariamente malos**, para todo k fijo (Papadimitriou y Steiglitz, 1978).
- **Calidad, en la práctica:** en instancias uniformes en el plano, 2-opt termina $\approx 5\,\%$ sobre la cota de Held–Karp ($\approx$ el óptimo); 3-opt, $\approx 3\,\%$; Lin–Kernighan, $\approx 2\,\%$ (Johnson y McGeoch, 1997). Con $n = 10^5$, en segundos.

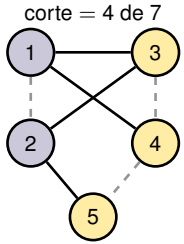
¿Cuánto cuesta y cuánto vale 2-opt?

- **Por iteración:** $\Theta(n^2)$ vecinas, $O(1)$ cada una. Con «listas de vecinos» (los k más cercanos de cada ciudad), $O(kn)$ en la práctica.
- **Iteraciones:** en el peor caso, **exponenciales** en n, incluso en el plano (Englert, Röglin y Vöcking, 2014). En la práctica, pocas pasadas.
- **Calidad, en general:** sin desigualdad triangular hay instancias donde los óptimos locales de k-opt son **arbitrariamente malos**, para todo k fijo (Papadimitriou y Steiglitz, 1978).
- **Calidad, en la práctica:** en instancias uniformes en el plano, 2-opt termina $\approx 5\,\%$ sobre la cota de Held–Karp ($\approx$ el óptimo); 3-opt, $\approx 3\,\%$; Lin–Kernighan, $\approx 2\,\%$ (Johnson y McGeoch, 1997). Con $n = 10^5$, en segundos.

Moraleja: búsqueda local es *la* herramienta práctica para problemas difíciles grandes. Pero lo que garantiza (un óptimo local) es muy poco, y hay que **medir**.

Una búsqueda local con garantía: Max-Cut

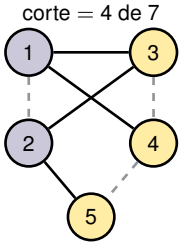
Problema: dado $G = (V, E)$, partir V en (A, B) maximizando las aristas con un extremo en cada lado (el *corte*). NP-difícil.



Una búsqueda local con garantía: Max-Cut

Problema: dado $G = (V, E)$, partir V en (A, B) maximizando las aristas con un extremo en cada lado (el *corte*). NP-difícil.

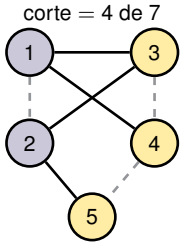
- **Vecindad:** cambiar *un* vértice de lado. Moverse si el corte crece.



Una búsqueda local con garantía: Max-Cut

Problema: dado $G = (V, E)$, partir V en (A, B) maximizando las aristas con un extremo en cada lado (el *corte*). NP-difícil.

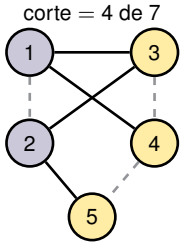
- **Vecindad:** cambiar *un* vértice de lado. Moverse si el corte crece.
- **Iteraciones:** a lo sumo $|E|$ (el corte es entero, crece en ≥ 1 y vale $\leq |E|$). **Polinomial.**



Una búsqueda local con garantía: Max-Cut

Problema: dado $G = (V, E)$, partir V en (A, B) maximizando las aristas con un extremo en cada lado (el *corte*). NP-difícil.

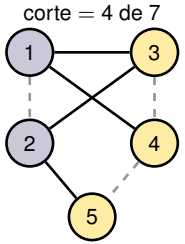
- **Vecindad:** cambiar *un* vértice de lado. Moverse si el corte crece.
- **Iteraciones:** a lo sumo $|E|$ (el corte es entero, crece en ≥ 1 y vale $\leq |E|$). **Polinomial.**
- **Garantía:** en un óptimo local, el corte tiene $\geq \frac{|E|}{2} \geq \frac{\mathrm{OPT}}{2}$ aristas. $\longrightarrow$ Pizarrón.



Una búsqueda local con garantía: Max-Cut

Problema: dado $G = (V, E)$, partir V en (A, B) maximizando las aristas con un extremo en cada lado (el *corte*). NP-difícil.

- **Vecindad:** cambiar *un* vértice de lado. Moverse si el corte crece.
- **Iteraciones:** a lo sumo $|E|$ (el corte es entero, crece en ≥ 1 y vale $\leq |E|$). **Polinomial.**
- **Garantía:** en un óptimo local, el corte tiene $\geq \frac{|E|}{2} \geq \frac{\text{OPT}}{2}$ aristas. $\longrightarrow$ Pizarrón.

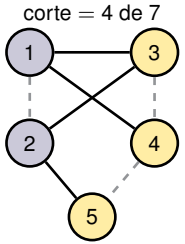


El vértice 4 tiene una arista cruzando y dos adentro: cambiarlo de lado da corte 5. En un óptimo local, *todo* vértice tiene al menos la mitad de sus aristas cruzando.

Una búsqueda local con garantía: Max-Cut

Problema: dado $G = (V, E)$, partir V en (A, B) maximizando las aristas con un extremo en cada lado (el *corte*). NP-difícil.

- **Vecindad:** cambiar *un* vértice de lado. Moverse si el corte crece.
- **Iteraciones:** a lo sumo $|E|$ (el corte es entero, crece en ≥ 1 y vale $\leq |E|$). **Polinomial.**
- **Garantía:** en un óptimo local, el corte tiene $\geq \frac{|E|}{2} \geq \frac{\text{OPT}}{2}$ aristas. $\longrightarrow$ Pizarrón.



El vértice 4 tiene una arista cruzando y dos adentro: cambiarlo de lado da corte 5. En un óptimo local, *todo* vértice tiene al menos la mitad de sus aristas cruzando.

Esto ya es un **algoritmo de aproximación** (sección 4): una búsqueda local *más una demostración*. La demostración usa sólo que estamos en un óptimo local, no cómo llegamos.

¿Qué es una heurística?

Definición (para esta materia; del griego *heurísko*, «encontrar», como *¡eureka!*)

Una **heurística** es un algoritmo que busca una solución factible *buena* de un problema de optimización, en tiempo razonable, **sin garantía** sobre la calidad de la solución que devuelve (ni, a veces, sobre encontrar alguna).

¿Qué es una heurística?

Definición (para esta materia; del griego *heurísko*, «encontrar», como *¡eureka!*)

Una **heurística** es un algoritmo que busca una solución factible *buena* de un problema de optimización, en tiempo razonable, **sin garantía** sobre la calidad de la solución que devuelve (ni, a veces, sobre encontrar alguna).

- Dos familias: **constructivas** (arman la solución de a una decisión: *greedy*) y **de mejora** (modifican una solución dada: *búsqueda local*).

¿Qué es una heurística?

Definición (para esta materia; del griego *heurísko*, «encontrar», como *¡eureka!*)

Una **heurística** es un algoritmo que busca una solución factible *buena* de un problema de optimización, en tiempo razonable, **sin garantía** sobre la calidad de la solución que devuelve (ni, a veces, sobre encontrar alguna).

- Dos familias: **constructivas** (arman la solución de a una decisión: *greedy*) y **de mejora** (modifican una solución dada: *búsqueda local*).
- «Sin garantía» no significa «mala»: LKH para el TSP es una heurística y es lo mejor que existe. Significa que **no hay un teorema** que acote la distancia al óptimo.

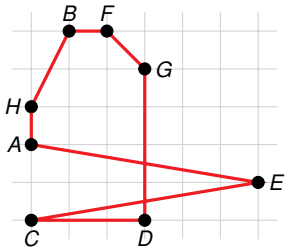
¿Qué es una heurística?

Definición (para esta materia; del griego *heurísko*, «encontrar», como *¡eureka!*)

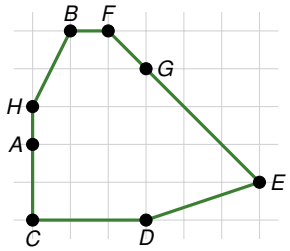
Una **heurística** es un algoritmo que busca una solución factible *buena* de un problema de optimización, en tiempo razonable, **sin garantía** sobre la calidad de la solución que devuelve (ni, a veces, sobre encontrar alguna).

- Dos familias: **constructivas** (arman la solución de a una decisión: *greedy*) y **de mejora** (modifican una solución dada: *búsqueda local*).
- «Sin garantía» no significa «mala»: LKH para el TSP es una heurística y es lo mejor que existe. Significa que **no hay un teorema** que acote la distancia al óptimo.
- Un **greedy con demostración** (clase 10: intercambio + inducción) es un algoritmo *exacto*. Un **greedy sin demostración** es una heurística. La diferencia no está en el código: está en lo que podemos afirmar.

TSP: vecino más cercano

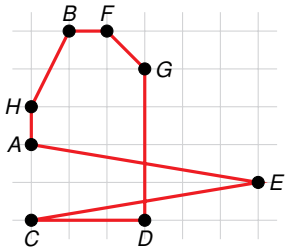


Vecino más cercano desde *A*: *AHBFGDCE*, costo 24,82.

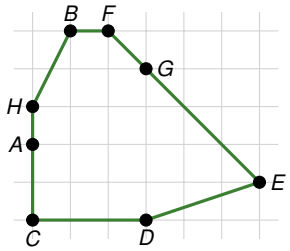


Óptimo: *AHBFGEDC*, costo 18,06.

TSP: vecino más cercano



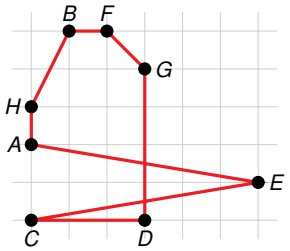
Vecino más cercano desde *A*: *AHBFGDCE*, costo 24,82.



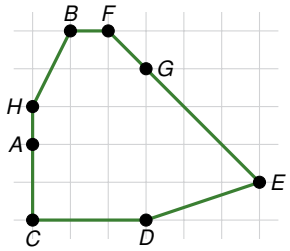
Óptimo: *AHBFGEDC*, costo 18,06.

- **Vecino más cercano:** ir a la ciudad no visitada más cercana; al final, volver. $O(n^2)$.

TSP: vecino más cercano



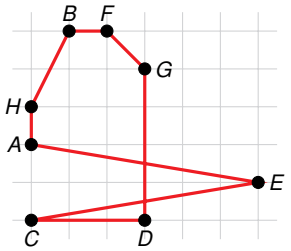
Vecino más cercano desde A: $AHBFGDCE$, costo 24,82.



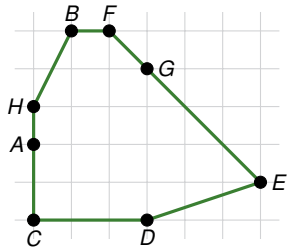
Óptimo: $AHBFGEDC$, costo 18,06.

- **Vecino más cercano:** ir a la ciudad no visitada más cercana; al final, volver. $O(n^2)$.
- Las primeras decisiones son buenas y baratas; las últimas (volver por lo que quedó lejos) son carísimas. Acá: 37 % sobre el óptimo, y con cruces.

TSP: vecino más cercano



Vecino más cercano desde A: $AHBFGDCE$, costo 24,82.



Óptimo: $AHBFGEDC$, costo 18,06.

- **Vecino más cercano:** ir a la ciudad no visitada más cercana; al final, volver. $O(n^2)$.
- Las primeras decisiones son buenas y baratas; las últimas (volver por lo que quedó lejos) son carísimas. Acá: 37 % sobre el óptimo, y con cruces.
- **Teorema** (Rosenkrantz, Stearns y Lewis, 1977): hay instancias, incluso métricas, donde da $\Theta(\log n)$ veces el óptimo. Sin desigualdad triangular, arbitrariamente peor.

Más heurísticas constructivas (y dónde termina la garantía)

Problema	Heurística greedy	¿Garantía?
Coloreo	Recorrer los vértices en algún orden; dar a cada uno el menor color libre.	$\leq \Delta + 1$ colores. *Existe* un orden que da el óptimo; encontrarlo es tan difícil como el problema.
Clique / conj. indep.	Agregar el vértice de mayor (menor) grado compatible.	Ninguna. Y no la hay: clique no admite factor $n^{1-\varepsilon}$ si P $\neq$ NP (Håstad 1999, Zuckerman 2007).
Vertex cover	Agregar el vértice de mayor grado; borrar sus aristas; repetir.	$\Theta(\log n)$ en el peor caso. *No* es una 2-aproximación (sección 4).
Mochila 0/1	Ordenar por v_i/w_i; agregar mientras entre.	Arbitrariamente malo. Con la corrección «máx(greedy, mejor ítem solo)», **2-aproximación**.

Más heurísticas constructivas (y dónde termina la garantía)

Problema	Heurística greedy	¿Garantía?
Coloreo	Recorrer los vértices en algún orden; dar a cada uno el menor color libre.	$\leq \Delta + 1$ colores. *Existe* un orden que da el óptimo; encontrarlo es tan difícil como el problema.
Clique / conj. indep.	Agregar el vértice de mayor (menor) grado compatible.	Ninguna. Y no la hay: clique no admite factor $n^{1-\varepsilon}$ si P $\neq$ NP (Håstad 1999, Zuckerman 2007).
Vertex cover	Agregar el vértice de mayor grado; borrar sus aristas; repetir.	$\Theta(\log n)$ en el peor caso. *No* es una 2-aproximación (sección 4).
Mochila 0/1	Ordenar por v_i/w_i; agregar mientras entre.	Arbitrariamente malo. Con la corrección «máx(greedy, mejor ítem solo)», **2-aproximación**.

La misma idea (greedy) puede ser un algoritmo exacto (mochila fraccionaria), una heurística sin garantía (mochila 0/1) o una aproximación (mochila 0/1 corregida). **La etiqueta la pone el teorema, no el código.**

Cómo se evalúa una heurística

Sin teorema de peor caso, lo que queda es el **experimento**. Uno serio tiene:

1. **Instancias:** un conjunto representativo, de varios tamaños; idealmente públicas (TSPLIB, DIMACS).

Cómo se evalúa una heurística

Sin teorema de peor caso, lo que queda es el **experimento**. Uno serio tiene:

1. **Instancias:** un conjunto representativo, de varios tamaños; idealmente públicas (TSPLIB, DIMACS).
2. **Una referencia:** el óptimo cuando se lo puede calcular (instancias chicas, algoritmo exacto); si no, una **cota inferior** (TSP: árbol generador mínimo) o la mejor solución conocida.

Cómo se evalúa una heurística

Sin teorema de peor caso, lo que queda es el **experimento**. Uno serio tiene:

1. **Instancias:** un conjunto representativo, de varios tamaños; idealmente públicas (TSPLIB, DIMACS).
2. **Una referencia:** el óptimo cuando se lo puede calcular (instancias chicas, algoritmo exacto); si no, una **cota inferior** (TSP: árbol generador mínimo) o la mejor solución conocida.
3. **Métricas:** $gap = \frac{c(\text{heurística}) - c(\text{referencia})}{c(\text{referencia})}$ y tiempo. Se compara a *igual tiempo*, no a igual cantidad de iteraciones.

Cómo se evalúa una heurística

Sin teorema de peor caso, lo que queda es el **experimento**. Uno serio tiene:

1. **Instancias:** un conjunto representativo, de varios tamaños; idealmente públicas (TSPLIB, DIMACS).
2. **Una referencia:** el óptimo cuando se lo puede calcular (instancias chicas, algoritmo exacto); si no, una **cota inferior** (TSP: árbol generador mínimo) o la mejor solución conocida.
3. **Métricas:** $gap = \frac{c(\text{heurística}) - c(\text{referencia})}{c(\text{referencia})}$ y tiempo. Se compara a *igual tiempo*, no a igual cantidad de iteraciones.
4. **Repeticiones:** si es randomizada, varias corridas con distintas semillas; reportar media, desviación y mejor, no sólo el mejor.

Cómo se evalúa una heurística

Sin teorema de peor caso, lo que queda es el **experimento**. Uno serio tiene:

1. **Instancias:** un conjunto representativo, de varios tamaños; idealmente públicas (TSPLIB, DIMACS).
2. **Una referencia:** el óptimo cuando se lo puede calcular (instancias chicas, algoritmo exacto); si no, una **cota inferior** (TSP: árbol generador mínimo) o la mejor solución conocida.
3. **Métricas:** $gap = \frac{c(\text{heurística}) - c(\text{referencia})}{c(\text{referencia})}$ y tiempo. Se compara a *igual tiempo*, no a igual cantidad de iteraciones.
4. **Repeticiones:** si es randomizada, varias corridas con distintas semillas; reportar media, desviación y mejor, no sólo el mejor.
5. **Comparaciones honestas:** contra la versión trivial (greedy solo, búsqueda local sola), con parámetros calibrados en instancias *distintas* de las reportadas.

Cómo se evalúa una heurística

Sin teorema de peor caso, lo que queda es el **experimento**. Uno serio tiene:

1. **Instancias:** un conjunto representativo, de varios tamaños; idealmente públicas (TSPLIB, DIMACS).
2. **Una referencia:** el óptimo cuando se lo puede calcular (instancias chicas, algoritmo exacto); si no, una **cota inferior** (TSP: árbol generador mínimo) o la mejor solución conocida.
3. **Métricas:** $gap = \frac{c(\text{heurística}) - c(\text{referencia})}{c(\text{referencia})}$ y tiempo. Se compara a *igual tiempo*, no a igual cantidad de iteraciones.
4. **Repeticiones:** si es randomizada, varias corridas con distintas semillas; reportar media, desviación y mejor, no sólo el mejor.
5. **Comparaciones honestas:** contra la versión trivial (greedy solo, búsqueda local sola), con parámetros calibrados en instancias *distintas* de las reportadas.

Esto es, palabra por palabra, lo que se espera del TP de heurísticas.

Las dos acepciones de «heurística»

La palabra se usa para dos cosas distintas, y en el parcial hay que saber cuál es cuál.

1. Un algoritmo heurístico

Un procedimiento completo que devuelve una solución factible sin garantía de calidad. Vecino más cercano, 2-opt, min-conflicts, GRASP.

Cede optimalidad.

Las dos acepciones de «heurística»

La palabra se usa para dos cosas distintas, y en el parcial hay que saber cuál es cuál.

1. Un algoritmo heurístico

Un procedimiento completo que devuelve una solución factible sin garantía de calidad. Vecino más cercano, 2-opt, min-conflicts, GRASP.

Cede optimalidad.

2. Una función heurística

Una **estimación** $h(\cdot)$ de lo que falta (para completar una parcial, o para llegar al objetivo), usada para *guiar* un algoritmo que sí tiene garantía.

No cede nada, si h cumple ciertas condiciones.

Las dos acepciones de «heurística»

La palabra se usa para dos cosas distintas, y en el parcial hay que saber cuál es cuál.

1. Un algoritmo heurístico

Un procedimiento completo que devuelve una solución factible sin garantía de calidad. Vecino más cercano, 2-opt, min-conflicts, GRASP.

Cede optimalidad.

2. Una función heurística

Una **estimación** $h(\cdot)$ de lo que falta (para completar una parcial, o para llegar al objetivo), usada para *guiar* un algoritmo que sí tiene garantía.

No cede nada, si h cumple ciertas condiciones.

La segunda ya la vieron: la **cota inferior** $L(p)$ de las podas por optimalidad de la clase 5 es una función heurística. Y es el corazón de A* (sección 6). El nombre es el mismo por razones históricas; los conceptos, no.

Algoritmos de aproximación

Definición

Sea Π un problema de *minimización* y $\rho \geq 1$. Un algoritmo $\mathcal{A}$ es una **ρ-aproximación** para Π si corre en **tiempo polinomial** y, para **toda** instancia I,

$$c(\mathcal{A}(I)) \leq \rho \cdot \mathrm{OPT}(I).$$

Para *maximización*: $c(\mathcal{A}(I)) \geq \mathrm{OPT}(I)/\rho$. ρ es el **factor** (o *razón*) de aproximación; puede depender de n.

Algoritmos de aproximación

Definición

Sea Π un problema de *minimización* y $\rho \geq 1$. Un algoritmo $\mathcal{A}$ es una **ρ-aproximación** para Π si corre en **tiempo polinomial** y, para **toda** instancia I,

$$c(\mathcal{A}(I)) \leq \rho \cdot \mathrm{OPT}(I).$$

Para *maximización*: $c(\mathcal{A}(I)) \geq \mathrm{OPT}(I)/\rho$. ρ es el **factor** (o *razón*) de aproximación; puede depender de n.

- Es una afirmación de **peor caso**, para toda instancia, con **demostración**. Exactamente lo que una heurística no tiene.

Algoritmos de aproximación

Definición

Sea Π un problema de *minimización* y $\rho \geq 1$. Un algoritmo $\mathcal{A}$ es una **ρ-aproximación** para Π si corre en **tiempo polinomial** y, para **toda** instancia I,

$$c(\mathcal{A}(I)) \leq \rho \cdot \mathrm{OPT}(I).$$

Para *maximización*: $c(\mathcal{A}(I)) \geq \mathrm{OPT}(I)/\rho$. ρ es el **factor** (o *razón*) de aproximación; puede depender de n.

- Es una afirmación de **peor caso**, para toda instancia, con **demostración**. Exactamente lo que una heurística no tiene.
- $\rho = 1$: exacto. $\rho = 2$: «a lo sumo el doble del óptimo». $\rho = O(\log n)$: también cuenta.

Algoritmos de aproximación

Definición

Sea Π un problema de *minimización* y $\rho \geq 1$. Un algoritmo $\mathcal{A}$ es una **ρ-aproximación** para Π si corre en **tiempo polinomial** y, para **toda** instancia I,

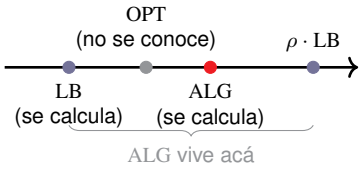
$$c(\mathcal{A}(I)) \leq \rho \cdot \mathrm{OPT}(I).$$

Para *maximización*: $c(\mathcal{A}(I)) \geq \mathrm{OPT}(I)/\rho$. ρ es el **factor** (o *razón*) de aproximación; puede depender de n.

- Es una afirmación de **peor caso**, para toda instancia, con **demostración**. Exactamente lo que una heurística no tiene.
- $\rho = 1$: exacto. $\rho = 2$: «a lo sumo el doble del óptimo». $\rho = O(\log n)$: también cuenta.
- Sigue siendo polinomial: no cedimos rapidez ni generalidad. Cedimos *exactamente* un factor ρ de optimalidad, y sabemos cuál.

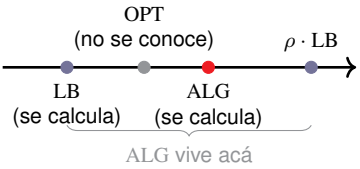
El truco: comparar contra una cota

- No conocemos $\mathrm{OPT}(I)$: calcularlo es el problema difícil.



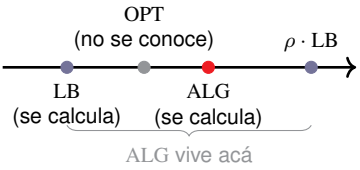
El truco: comparar contra una cota

- No conocemos $\mathrm{OPT}(I)$: calcularlo es el problema difícil.
- Lo que sí podemos: una cantidad $\mathrm{LB}(I)$ que **sepamos demostrar** que cumple $\mathrm{LB}(I) \leq \mathrm{OPT}(I)$.



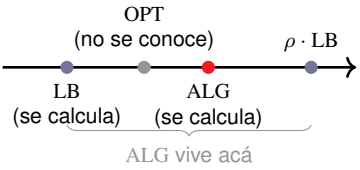
El truco: comparar contra una cota

- No conocemos $\mathrm{OPT}(I)$: calcularlo es el problema difícil.
- Lo que sí podemos: una cantidad $\mathrm{LB}(I)$ que **sepamos demostrar** que cumple $\mathrm{LB}(I) \leq \mathrm{OPT}(I)$.
- Y después demostrar $c(\mathcal{A}(I)) \leq \rho \cdot \mathrm{LB}(I)$. Encadenando: $\mathrm{ALG} \leq \rho \cdot \mathrm{LB} \leq \rho \cdot \mathrm{OPT}$.



El truco: comparar contra una cota

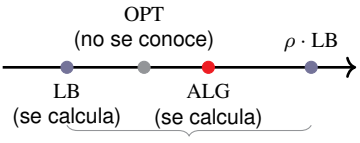
- No conocemos $\mathrm{OPT}(I)$: calcularlo es el problema difícil.
- Lo que sí podemos: una cantidad $\mathrm{LB}(I)$ que **sepamos demostrar** que cumple $\mathrm{LB}(I) \leq \mathrm{OPT}(I)$.
- Y después demostrar $c(\mathcal{A}(I)) \leq \rho \cdot \mathrm{LB}(I)$. Encadenando: $\mathrm{ALG} \leq \rho \cdot \mathrm{LB} \leq \rho \cdot \mathrm{OPT}$.



- Toda demostración de aproximación tiene esta forma. La creatividad está en elegir la cota.

El truco: comparar contra una cota

- No conocemos $\mathrm{OPT}(I)$: calcularlo es el problema difícil.
- Lo que sí podemos: una cantidad $\mathrm{LB}(I)$ que **sepamos demostrar** que cumple $\mathrm{LB}(I) \leq \mathrm{OPT}(I)$.
- Y después demostrar $c(\mathcal{A}(I)) \leq \rho \cdot \mathrm{LB}(I)$. Encadenando: $\mathrm{ALG} \leq \rho \cdot \mathrm{LB} \leq \rho \cdot \mathrm{OPT}$.

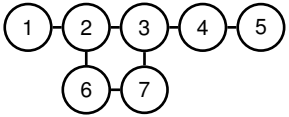


- Toda demostración de aproximación tiene esta forma. La creatividad está en elegir la cota.
- Es la **misma idea** que las podas por optimalidad (clase 5: relajar para acotar) y que el gap de la sección anterior. Y sirve en concreto: si $\mathrm{ALG} \leq 1{,}05 \cdot \mathrm{LB}$ en *esta* instancia, estamos a 5 % del óptimo aunque no lo conozcamos.

Vertex cover: 2-aproximación por matching maximal

Problema: hallar $C \subseteq V$ mínimo tal que toda arista tiene un extremo en C. NP-difícil.

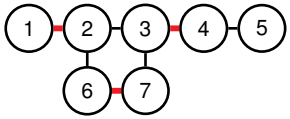
function $\mathrm{aproxVC}(G)$
- $C \leftarrow \emptyset$; $E' \leftarrow E$;
- **while** $E' \neq \emptyset$ **do**
  - elegir cualquier $\{u, v\} \in E'$;
  - $C \leftarrow C \cup \{u, v\}$;
  - borrar de E' toda arista incidente a u o a v;
- **return** C;



Vertex cover: 2-aproximación por matching maximal

Problema: hallar $C \subseteq V$ mínimo tal que toda arista tiene un extremo en C. NP-difícil.

function $\mathrm{aproxVC}(G)$
- $C \leftarrow \emptyset$; $E' \leftarrow E$;
- **while** $E' \neq \emptyset$ **do**
  - elegir cualquier $\{u, v\} \in E'$;
  - $C \leftarrow C \cup \{u, v\}$;
  - borrar de E' toda arista incidente a u o a v;
- **return** C;

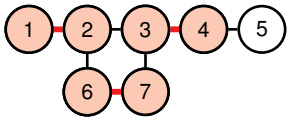


Las aristas elegidas forman un **matching maximal** M: disjuntas, y toda otra arista toca alguna.

Vertex cover: 2-aproximación por matching maximal

Problema: hallar $C \subseteq V$ mínimo tal que toda arista tiene un extremo en C. NP-difícil.

function $\mathrm{aproxVC}(G)$
- $C \leftarrow \emptyset$; $E' \leftarrow E$;
- **while** $E' \neq \emptyset$ **do**
  - elegir cualquier $\{u, v\} \in E'$;
  - $C \leftarrow C \cup \{u, v\}$;
  - borrar de E' toda arista incidente a u o a v;
- **return** C;

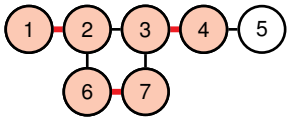


Las aristas elegidas forman un **matching maximal** M: disjuntas, y toda otra arista toca alguna. C = sus 6 extremos; el óptimo es $\{2, 4, 7\}$: $|C| = 2 \cdot \mathrm{OPT}$, justo.

Vertex cover: 2-aproximación por matching maximal

Problema: hallar $C \subseteq V$ mínimo tal que toda arista tiene un extremo en C. NP-difícil.

function $\mathrm{aproxVC}(G)$
- $C \leftarrow \emptyset$; $E' \leftarrow E$;
- **while** $E' \neq \emptyset$ **do**
  - elegir cualquier $\{u, v\} \in E'$;
  - $C \leftarrow C \cup \{u, v\}$;
  - borrar de E' toda arista incidente a u o a v;
- **return** C;



Las aristas elegidas forman un **matching maximal** M: disjuntas, y toda otra arista toca alguna. C = sus 6 extremos; el óptimo es $\{2, 4, 7\}$: $|C| = 2 \cdot \mathrm{OPT}$, justo.

$|C| = 2|M|$. Para la 2-aproximación hace falta: **(a)** C es un cubrimiento; **(b)** $\mathrm{OPT} \geq |M|$. La cota es el matching.

Vertex cover: la demostración

Teorema. $\mathrm{aproxVC}$ es una 2-aproximación para vertex cover.

- **Polinomial:** cada iteración borra al menos una arista; $O(|V| + |E|)$.

Vertex cover: la demostración

Teorema. $\mathrm{aproxVC}$ es una 2-aproximación para vertex cover.

- **Polinomial:** cada iteración borra al menos una arista; $O(|V| + |E|)$.
- C **es un cubrimiento:** una arista sale de E' sólo cuando un extremo suyo entra en C, y el ciclo termina con $E' = \emptyset$.

Vertex cover: la demostración

Teorema. $\mathrm{aproxVC}$ es una 2-aproximación para vertex cover.

- **Polinomial:** cada iteración borra al menos una arista; $O(|V| + |E|)$.
- C **es un cubrimiento:** una arista sale de E' sólo cuando un extremo suyo entra en C, y el ciclo termina con $E' = \emptyset$.
- **La cota:** las aristas elegidas, M, son disjuntas dos a dos (al elegir $\{u, v\}$ se borran todas las incidentes a u y a v). Todo cubrimiento C^* contiene un extremo de *cada* arista de M, y esos extremos son **distintos**: $|C^*| \geq |M|$. En particular $\mathrm{OPT} \geq |M|$.

Vertex cover: la demostración

Teorema. $\mathrm{aproxVC}$ es una 2-aproximación para vertex cover.

- **Polinomial:** cada iteración borra al menos una arista; $O(|V| + |E|)$.
- C **es un cubrimiento:** una arista sale de E' sólo cuando un extremo suyo entra en C, y el ciclo termina con $E' = \emptyset$.
- **La cota:** las aristas elegidas, M, son disjuntas dos a dos (al elegir $\{u, v\}$ se borran todas las incidentes a u y a v). Todo cubrimiento C^* contiene un extremo de *cada* arista de M, y esos extremos son **distintos**: $|C^*| \geq |M|$. En particular $\mathrm{OPT} \geq |M|$.
- **Cierre:** $|C| = 2|M| \leq 2 \cdot \mathrm{OPT}$. $\longrightarrow$ Pizarrón.

Vertex cover: la demostración

Teorema. $\mathrm{aproxVC}$ es una 2-aproximación para vertex cover.

- **Polinomial:** cada iteración borra al menos una arista; $O(|V| + |E|)$.
- C **es un cubrimiento:** una arista sale de E' sólo cuando un extremo suyo entra en C, y el ciclo termina con $E' = \emptyset$.
- **La cota:** las aristas elegidas, M, son disjuntas dos a dos (al elegir $\{u, v\}$ se borran todas las incidentes a u y a v). Todo cubrimiento C^* contiene un extremo de *cada* arista de M, y esos extremos son **distintos**: $|C^*| \geq |M|$. En particular $\mathrm{OPT} \geq |M|$.
- **Cierre:** $|C| = 2|M| \leq 2 \cdot \mathrm{OPT}$. $\longrightarrow$ Pizarrón.

- El ejemplo anterior muestra que el 2 es **ajustado** para este algoritmo.

Vertex cover: la demostración

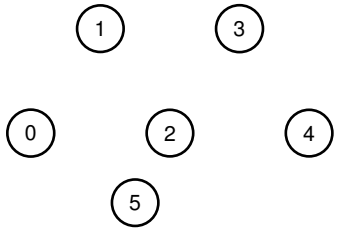
Teorema. $\mathrm{aproxVC}$ es una 2-aproximación para vertex cover.

- **Polinomial:** cada iteración borra al menos una arista; $O(|V| + |E|)$.
- C **es un cubrimiento:** una arista sale de E' sólo cuando un extremo suyo entra en C, y el ciclo termina con $E' = \emptyset$.
- **La cota:** las aristas elegidas, M, son disjuntas dos a dos (al elegir $\{u, v\}$ se borran todas las incidentes a u y a v). Todo cubrimiento C^* contiene un extremo de *cada* arista de M, y esos extremos son **distintos**: $|C^*| \geq |M|$. En particular $\mathrm{OPT} \geq |M|$.
- **Cierre:** $|C| = 2|M| \leq 2 \cdot \mathrm{OPT}$. $\longrightarrow$ Pizarrón.

- El ejemplo anterior muestra que el 2 es **ajustado** para este algoritmo.
- ¿Otro algoritmo con factor < 2? Abierto; se sabe que aproximar por debajo de $\approx 1{,}41$ es NP-difícil (Khot, Minzer y Safra, 2018). El greedy por grado, que parece más inteligente, es $\Theta(\log n)$.

TSP métrico: 2-aproximación por árbol generador mínimo

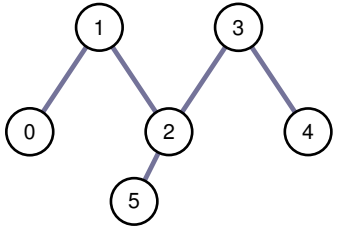
TSP métrico: las distancias cumplen la **desigualdad triangular**, $d(u, w) \leq d(u, v) + d(v, w)$ (distancias en el plano, en una red de rutas...). Sigue siendo NP-difícil.



TSP métrico: 2-aproximación por árbol generador mínimo

TSP métrico: las distancias cumplen la **desigualdad triangular**, $d(u, w) \leq d(u, v) + d(v, w)$ (distancias en el plano, en una red de rutas...). Sigue siendo NP-difícil.

1. Calcular un árbol generador mínimo T (Prim o Kruskal).

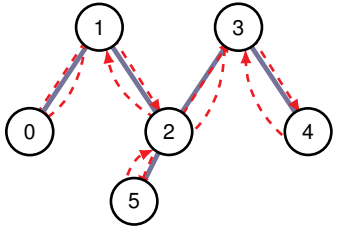


AGM T, costo 16,66.

TSP métrico: 2-aproximación por árbol generador mínimo

TSP métrico: las distancias cumplen la **desigualdad triangular**, $d(u, w) \leq d(u, v) + d(v, w)$ (distancias en el plano, en una red de rutas...). Sigue siendo NP-difícil.

1. Calcular un árbol generador mínimo T (Prim o Kruskal).
2. Recorrer T en preorden (DFS): la lista H de los vértices en el orden en que se los *descubre*.

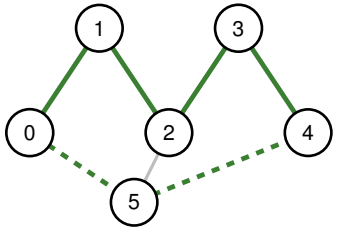


Recorrido de Euler (cada arista dos veces): 0 1 2 3 4 3 2 5 2 1 0, costo 33,32.

TSP métrico: 2-aproximación por árbol generador mínimo

TSP métrico: las distancias cumplen la **desigualdad triangular**, $d(u, w) \leq d(u, v) + d(v, w)$ (distancias en el plano, en una red de rutas...). Sigue siendo NP-difícil.

1. Calcular un árbol generador mínimo T (Prim o Kruskal).
2. Recorrer T en preorden (DFS): la lista H de los vértices en el orden en que se los *descubre*.
3. Devolver el tour H.



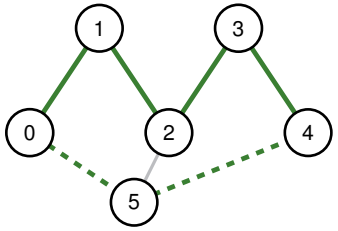
Preorden: 0 1 2 3 4 5. Tour con *atajos* (punteados): 23,41. Óptimo: 0 1 3 4 2 5, 21,05.

TSP métrico: 2-aproximación por árbol generador mínimo

TSP métrico: las distancias cumplen la **desigualdad triangular**, $d(u, w) \leq d(u, v) + d(v, w)$ (distancias en el plano, en una red de rutas...). Sigue siendo NP-difícil.

1. Calcular un árbol generador mínimo T (Prim o Kruskal).
2. Recorrer T en preorden (DFS): la lista H de los vértices en el orden en que se los *descubre*.
3. Devolver el tour H.

Polinomial: $O(n^2)$ sobre K_n.

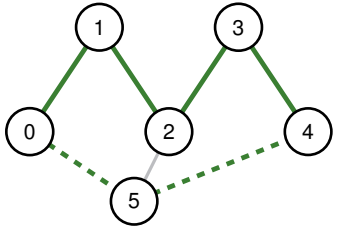


Preorden: 0 1 2 3 4 5. Tour con *atajos* (punteados): 23,41. Óptimo: 0 1 3 4 2 5, 21,05.

TSP métrico: 2-aproximación por árbol generador mínimo

TSP métrico: las distancias cumplen la **desigualdad triangular**, $d(u, w) \leq d(u, v) + d(v, w)$ (distancias en el plano, en una red de rutas...). Sigue siendo NP-difícil.

1. Calcular un árbol generador mínimo T (Prim o Kruskal).
2. Recorrer T en preorden (DFS): la lista H de los vértices en el orden en que se los *descubre*.
3. Devolver el tour H.



Polinomial: $O(n^2)$ sobre K_n.

Preorden: 0 1 2 3 4 5. Tour con *atajos* (punteados): 23,41. Óptimo: 0 1 3 4 2 5, 21,05.

Afirmación: $c(H) \leq 2 \cdot \mathrm{OPT}$. La cota es el árbol: $\mathrm{OPT} \geq c(T)$.

TSP métrico: la demostración

Teorema. En el TSP métrico, el tour H del preorden de un árbol generador mínimo cumple $c(H) \leq 2 \cdot \mathrm{OPT}$.

1. **Cota:** $\mathrm{OPT} \geq c(T)$. A un tour óptimo H^* le quitamos una arista: queda un camino que pasa por todos los vértices, un árbol generador. Como T es mínimo, $c(T) \leq c(H^*) = \mathrm{OPT}$.

TSP métrico: la demostración

Teorema. En el TSP métrico, el tour H del preorden de un árbol generador mínimo cumple $c(H) \leq 2 \cdot \mathrm{OPT}$.

1. **Cota:** $\mathrm{OPT} \geq c(T)$. A un tour óptimo H^* le quitamos una arista: queda un camino que pasa por todos los vértices, un árbol generador. Como T es mínimo, $c(T) \leq c(H^*) = \mathrm{OPT}$.
2. **Recorrido de Euler:** el recorrido W del DFS (bajar por cada arista y volver por ella) usa cada arista de T exactamente dos veces: $c(W) = 2 \cdot c(T)$.

TSP métrico: la demostración

Teorema. En el TSP métrico, el tour H del preorden de un árbol generador mínimo cumple $c(H) \leq 2 \cdot \mathrm{OPT}$.

1. **Cota:** $\mathrm{OPT} \geq c(T)$. A un tour óptimo H^* le quitamos una arista: queda un camino que pasa por todos los vértices, un árbol generador. Como T es mínimo, $c(T) \leq c(H^*) = \mathrm{OPT}$.
2. **Recorrido de Euler:** el recorrido W del DFS (bajar por cada arista y volver por ella) usa cada arista de T exactamente dos veces: $c(W) = 2 \cdot c(T)$.
3. **Atajos:** H se obtiene de W borrando cada aparición repetida de un vértice. Cada borrado reemplaza un tramo $u \to v_1 \to \cdots \to v_k \to w$ por la arista $u \to w$, y por la desigualdad triangular $d(u, w) \leq d(u, v_1) + \cdots + d(v_k, w)$. Luego $c(H) \leq c(W)$.

TSP métrico: la demostración

Teorema. En el TSP métrico, el tour H del preorden de un árbol generador mínimo cumple $c(H) \leq 2 \cdot \mathrm{OPT}$.

1. **Cota:** $\mathrm{OPT} \geq c(T)$. A un tour óptimo H^* le quitamos una arista: queda un camino que pasa por todos los vértices, un árbol generador. Como T es mínimo, $c(T) \leq c(H^*) = \mathrm{OPT}$.
2. **Recorrido de Euler:** el recorrido W del DFS (bajar por cada arista y volver por ella) usa cada arista de T exactamente dos veces: $c(W) = 2 \cdot c(T)$.
3. **Atajos:** H se obtiene de W borrando cada aparición repetida de un vértice. Cada borrado reemplaza un tramo $u \to v_1 \to \cdots \to v_k \to w$ por la arista $u \to w$, y por la desigualdad triangular $d(u, w) \leq d(u, v_1) + \cdots + d(v_k, w)$. Luego $c(H) \leq c(W)$.
4. **Cierre:** $c(H) \leq c(W) = 2 \cdot c(T) \leq 2 \cdot \mathrm{OPT}$. $\longrightarrow$ Pizarrón.

TSP métrico: la demostración

Teorema. En el TSP métrico, el tour H del preorden de un árbol generador mínimo cumple $c(H) \leq 2 \cdot \mathrm{OPT}$.

1. **Cota:** $\mathrm{OPT} \geq c(T)$. A un tour óptimo H^* le quitamos una arista: queda un camino que pasa por todos los vértices, un árbol generador. Como T es mínimo, $c(T) \leq c(H^*) = \mathrm{OPT}$.
2. **Recorrido de Euler:** el recorrido W del DFS (bajar por cada arista y volver por ella) usa cada arista de T exactamente dos veces: $c(W) = 2 \cdot c(T)$.
3. **Atajos:** H se obtiene de W borrando cada aparición repetida de un vértice. Cada borrado reemplaza un tramo $u \to v_1 \to \cdots \to v_k \to w$ por la arista $u \to w$, y por la desigualdad triangular $d(u, w) \leq d(u, v_1) + \cdots + d(v_k, w)$. Luego $c(H) \leq c(W)$.
4. **Cierre:** $c(H) \leq c(W) = 2 \cdot c(T) \leq 2 \cdot \mathrm{OPT}$. $\longrightarrow$ Pizarrón.

Mejor: Christofides (1976) agrega a T un matching perfecto mínimo entre sus vértices de grado impar: $\rho = 3/2$; récord por 45 años, hasta $3/2 - \varepsilon$ (Karlin, Klein y Oveis Gharan, 2021). En el plano, $(1 + \varepsilon)$ para todo ε (Arora 1998, Mitchell 1999).

¿Y sin desigualdad triangular?

Teorema. Si P $\neq$ NP, no existe ninguna ρ-aproximación polinomial para el TSP general, para *ningún* $\rho \geq 1$ constante.

Idea (reducción desde *ciclo hamiltoniano*, NP-completo):

¿Y sin desigualdad triangular?

Teorema. Si P $\neq$ NP, no existe ninguna ρ-aproximación polinomial para el TSP general, para *ningún* $\rho \geq 1$ constante.

Idea (reducción desde *ciclo hamiltoniano*, NP-completo):

- Dado $G = (V, E)$ con n vértices, armamos un TSP sobre K_n: $d(u, v) = 1$ si $\{u, v\} \in E$, y $d(u, v) = \rho n + 1$ si no.

¿Y sin desigualdad triangular?

Teorema. Si P $\neq$ NP, no existe ninguna ρ-aproximación polinomial para el TSP general, para *ningún* $\rho \geq 1$ constante.

Idea (reducción desde *ciclo hamiltoniano*, NP-completo):

- Dado $G = (V, E)$ con n vértices, armamos un TSP sobre K_n: $d(u, v) = 1$ si $\{u, v\} \in E$, y $d(u, v) = \rho n + 1$ si no.
- Si G tiene un ciclo hamiltoniano, ese ciclo es un tour de costo n: $\mathrm{OPT} = n$.

¿Y sin desigualdad triangular?

Teorema. Si P $\neq$ NP, no existe ninguna ρ-aproximación polinomial para el TSP general, para *ningún* $\rho \geq 1$ constante.

Idea (reducción desde *ciclo hamiltoniano*, NP-completo):

- Dado $G = (V, E)$ con n vértices, armamos un TSP sobre K_n: $d(u, v) = 1$ si $\{u, v\} \in E$, y $d(u, v) = \rho n + 1$ si no.
- Si G tiene un ciclo hamiltoniano, ese ciclo es un tour de costo n: $\mathrm{OPT} = n$.
- Si no, todo tour usa alguna arista de peso $\rho n + 1$: $\mathrm{OPT} \geq \rho n + 1 + (n - 1) > \rho n$.

¿Y sin desigualdad triangular?

Teorema. Si P $\neq$ NP, no existe ninguna ρ-aproximación polinomial para el TSP general, para *ningún* $\rho \geq 1$ constante.

Idea (reducción desde *ciclo hamiltoniano*, NP-completo):

- Dado $G = (V, E)$ con n vértices, armamos un TSP sobre K_n: $d(u, v) = 1$ si $\{u, v\} \in E$, y $d(u, v) = \rho n + 1$ si no.
- Si G tiene un ciclo hamiltoniano, ese ciclo es un tour de costo n: $\mathrm{OPT} = n$.
- Si no, todo tour usa alguna arista de peso $\rho n + 1$: $\mathrm{OPT} \geq \rho n + 1 + (n - 1) > \rho n$.
- Una ρ-aproximación devolvería costo $\leq \rho n$ en el primer caso y $> \rho n$ en el segundo: **decidiría ciclo hamiltoniano en tiempo polinomial**.

¿Y sin desigualdad triangular?

Teorema. Si P $\neq$ NP, no existe ninguna ρ-aproximación polinomial para el TSP general, para *ningún* $\rho \geq 1$ constante.

Idea (reducción desde *ciclo hamiltoniano*, NP-completo):

- Dado $G = (V, E)$ con n vértices, armamos un TSP sobre K_n: $d(u, v) = 1$ si $\{u, v\} \in E$, y $d(u, v) = \rho n + 1$ si no.
- Si G tiene un ciclo hamiltoniano, ese ciclo es un tour de costo n: $\mathrm{OPT} = n$.
- Si no, todo tour usa alguna arista de peso $\rho n + 1$: $\mathrm{OPT} \geq \rho n + 1 + (n - 1) > \rho n$.
- Una ρ-aproximación devolvería costo $\leq \rho n$ en el primer caso y $> \rho n$ en el segundo: **decidiría ciclo hamiltoniano en tiempo polinomial**.

$\longrightarrow$ Pizarrón. Moraleja: la desigualdad triangular se usó *una* vez (en los atajos) y sin ella el teorema es **falso**. Toda hipótesis que se usa hay que nombrarla.

Un catálogo (para tener el mapa)

Problema	Algoritmo	Factor	Referencia
Vertex cover	Matching maximal	2	Cormen 35.1
TSP métrico	AGM doblado / Christofides	2 / $3/2$	Cormen 35.2
TSP general	—	ninguno (si P $\neq$ NP)	Cormen 35.2.2
Balanceo de carga	Greedy / greedy ordenado	2 / $3/2$	K–T 11.1
Set cover	Greedy	$O(\ln n)$	K–T 11.3, Cormen 35.3
Max-Cut	Búsqueda local	2	K–T 12.4
Mochila 0/1	DP con redondeo (FPTAS)	$1 + \varepsilon$	K–T 11.8

Un catálogo (para tener el mapa)

Problema	Algoritmo	Factor	Referencia
Vertex cover	Matching maximal	2	Cormen 35.1
TSP métrico	AGM doblado / Christofides	2 / $3/2$	Cormen 35.2
TSP general	—	ninguno (si P $\neq$ NP)	Cormen 35.2.2
Balanceo de carga	Greedy / greedy ordenado	2 / $3/2$	K–T 11.1
Set cover	Greedy	$O(\ln n)$	K–T 11.3, Cormen 35.3
Max-Cut	Búsqueda local	2	K–T 12.4
Mochila 0/1	DP con redondeo (FPTAS)	$1+\varepsilon$	K–T 11.8

- **Esquemas:** un *PTAS* recibe $\varepsilon > 0$ y es $(1+\varepsilon)$-aproximación, polinomial para cada ε fijo; un *FPTAS* es polinomial en n *y* en $1/\varepsilon$ (mochila: $O(n^3/\varepsilon)$, redondeando valores y usando la DP de la clase 8).

Un catálogo (para tener el mapa)

Problema	Algoritmo	Factor	Referencia
Vertex cover	Matching maximal	2	Cormen 35.1
TSP métrico	AGM doblado / Christofides	2 / $3/2$	Cormen 35.2
TSP general	—	ninguno (si P $\neq$ NP)	Cormen 35.2.2
Balanceo de carga	Greedy / greedy ordenado	2 / $3/2$	K–T 11.1
Set cover	Greedy	$O(\ln n)$	K–T 11.3, Cormen 35.3
Max-Cut	Búsqueda local	2	K–T 12.4
Mochila 0/1	DP con redondeo (FPTAS)	$1+\varepsilon$	K–T 11.8

- **Esquemas:** un *PTAS* recibe $\varepsilon > 0$ y es $(1+\varepsilon)$-aproximación, polinomial para cada ε fijo; un *FPTAS* es polinomial en n *y* en $1/\varepsilon$ (mochila: $O(n^3/\varepsilon)$, redondeando valores y usando la DP de la clase 8).
- Para muchos problemas se conoce también una **cota de inaproximabilidad**: un factor por debajo del cual aproximar es NP-difícil (vertex cover: $\approx 1{,}41$; TSP general: cualquier constante).

Heurística vs. aproximación

	Heurística	Algoritmo de aproximación
Qué garantiza	Nada (a lo sumo, un óptimo local)	$\mathrm{ALG} \leq \rho \cdot \mathrm{OPT}$ en *toda* instancia
Cómo se justifica	Experimentos	Un teorema (y experimentos)
Calidad típica	Puede ser excelente (LKH: $<$ 0,1 %)	Puede ser mediocre (AGM doblado: 30–40 %)
Ejemplo	Vecino más cercano, 2-opt, GRASP	Matching maximal, AGM doblado

Heurística vs. aproximación

	Heurística	Algoritmo de aproximación
Qué garantiza	Nada (a lo sumo, un óptimo local)	$\mathrm{ALG} \leq \rho \cdot \mathrm{OPT}$ en *toda* instancia
Cómo se justifica	Experimentos	Un teorema (y experimentos)
Calidad típica	Puede ser excelente (LKH: $<$ 0,1 %)	Puede ser mediocre (AGM doblado: 30–40 %)
Ejemplo	Vecino más cercano, 2-opt, GRASP	Matching maximal, AGM doblado

- Un mismo algoritmo puede ser las dos cosas (búsqueda local para Max-Cut): lo que lo hace aproximación es el **teorema**. Y garantía de peor caso $\neq$ calidad típica.

Heurística vs. aproximación

	Heurística	Algoritmo de aproximación
Qué garantiza	Nada (a lo sumo, un óptimo local)	$\mathrm{ALG} \leq \rho \cdot \mathrm{OPT}$ en *toda* instancia
Cómo se justifica	Experimentos	Un teorema (y experimentos)
Calidad típica	Puede ser excelente (LKH: $<$ 0,1 %)	Puede ser mediocre (AGM doblado: 30–40 %)
Ejemplo	Vecino más cercano, 2-opt, GRASP	Matching maximal, AGM doblado

- Un mismo algoritmo puede ser las dos cosas (búsqueda local para Max-Cut): lo que lo hace aproximación es el **teorema**. Y garantía de peor caso $\neq$ calidad típica.
- «¿Es 2-opt un algoritmo de aproximación para el TSP?»

Heurística vs. aproximación

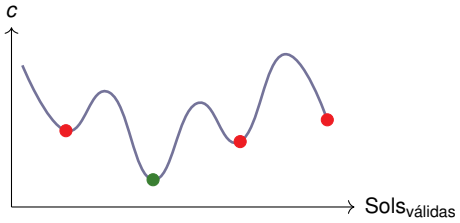
	Heurística	Algoritmo de aproximación
Qué garantiza	Nada (a lo sumo, un óptimo local)	$\mathrm{ALG} \leq \rho \cdot \mathrm{OPT}$ en *toda* instancia
Cómo se justifica	Experimentos	Un teorema (y experimentos)
Calidad típica	Puede ser excelente (LKH: $<$ 0,1 %)	Puede ser mediocre (AGM doblado: 30–40 %)
Ejemplo	Vecino más cercano, 2-opt, GRASP	Matching maximal, AGM doblado

- Un mismo algoritmo puede ser las dos cosas (búsqueda local para Max-Cut): lo que lo hace aproximación es el **teorema**. Y garantía de peor caso $\neq$ calidad típica.
- «¿Es 2-opt un algoritmo de aproximación para el TSP?» No: es una heurística; sin desigualdad triangular sus óptimos locales son arbitrariamente malos, y aun con ella no tiene factor constante (Chandra, Karloff y Tovey, 1999).

¿Descansamos 10 minutos?

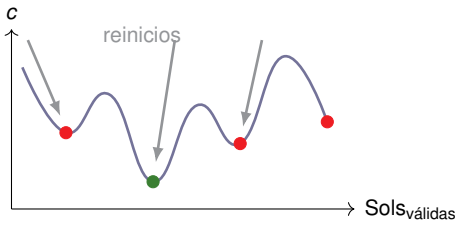


El problema de la búsqueda local, otra vez



La búsqueda local se queda en el primer óptimo local que encuentra. Tres formas de salir:

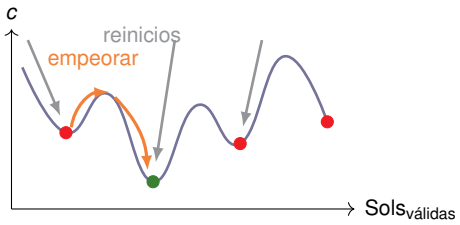
El problema de la búsqueda local, otra vez



La búsqueda local se queda en el primer óptimo local que encuentra. Tres formas de salir:

1. **Muchos comienzos:** reinicios al azar o construcciones distintas (multi-start, **GRASP**).

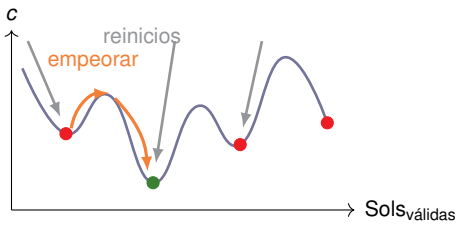
El problema de la búsqueda local, otra vez



La búsqueda local se queda en el primer óptimo local que encuentra. Tres formas de salir:

1. **Muchos comienzos:** reinicios al azar o construcciones distintas (multi-start, **GRASP**).
2. **Aceptar empeorar**, con control: a veces, al azar (*simulated annealing*); siempre, pero con memoria para no volver (*tabú*).

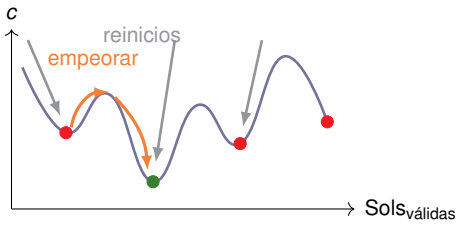
El problema de la búsqueda local, otra vez



La búsqueda local se queda en el primer óptimo local que encuentra. Tres formas de salir:

1. **Muchos comienzos:** reinicios al azar o construcciones distintas (multi-start, **GRASP**).
2. **Aceptar empeorar**, con control: a veces, al azar (*simulated annealing*); siempre, pero con memoria para no volver (*tabú*).
3. **Combinar soluciones:** poblaciones, cruza, mutación (*genéticos*).

El problema de la búsqueda local, otra vez



La búsqueda local se queda en el primer óptimo local que encuentra. Tres formas de salir:

1. **Muchos comienzos:** reinicios al azar o construcciones distintas (multi-start, **GRASP**).
2. **Aceptar empeorar**, con control: a veces, al azar (*simulated annealing*); siempre, pero con memoria para no volver (*tabú*).
3. **Combinar soluciones:** poblaciones, cruza, mutación (*genéticos*).

Cualquiera de las tres es una *estrategia* que se puede aplicar a cualquier problema. Eso tiene nombre:

¿Qué es una metaheurística?

Definición

Una **metaheurística** es una estrategia general, *independiente del problema*, que guía heurísticas subordinadas (construcción, búsqueda local) para explorar el espacio de soluciones más allá de un óptimo local. (El término es de Glover, 1986.)

¿Qué es una metaheurística?

Definición

Una **metaheurística** es una estrategia general, *independiente del problema*, que guía heurísticas subordinadas (construcción, búsqueda local) para explorar el espacio de soluciones más allá de un óptimo local. (El término es de Glover, 1986.)

- Es una *receta* para diseñar heurísticas, no una heurística: para usarla en un problema hay que instanciar la representación, la vecindad, el greedy, la evaluación.

¿Qué es una metaheurística?

Definición

Una **metaheurística** es una estrategia general, *independiente del problema*, que guía heurísticas subordinadas (construcción, búsqueda local) para explorar el espacio de soluciones más allá de un óptimo local. (El término es de Glover, 1986.)

- Es una *receta* para diseñar heurísticas, no una heurística: para usarla en un problema hay que instanciar la representación, la vecindad, el greedy, la evaluación.
- Todas balancean dos fuerzas: **intensificación** (explotar la zona donde hay buenas soluciones) y **diversificación** (explorar zonas nuevas).

¿Qué es una metaheurística?

Definición

Una **metaheurística** es una estrategia general, *independiente del problema*, que guía heurísticas subordinadas (construcción, búsqueda local) para explorar el espacio de soluciones más allá de un óptimo local. (El término es de Glover, 1986.)

- Es una *receta* para diseñar heurísticas, no una heurística: para usarla en un problema hay que instanciar la representación, la vecindad, el greedy, la evaluación.
- Todas balancean dos fuerzas: **intensificación** (explotar la zona donde hay buenas soluciones) y **diversificación** (explorar zonas nuevas).
- Ninguna garantiza nada sobre OPT. Todas tienen **parámetros** que hay que calibrar. Para todas vale lo de la sección 3: hay que *medir*.

Un zoológico

Familia	Metaheurística	Idea en una línea
Trayectoria	Multi-start / **GRASP**	Muchas construcciones greedy randomizadas + búsqueda local
	Simulated annealing	Aceptar empeoramientos con probabilidad $e^{-\Delta/T}$, T decreciente
	Búsqueda tabú	Siempre al mejor vecino, aunque empeore; prohibir deshacerlo
	Iterated local search	Óptimo local $\rightarrow$ perturbar $\rightarrow$ búsqueda local $\rightarrow$ aceptar o no
	VNS	Cambiar de vecindad cuando la actual se agota
Población	Algoritmos genéticos	Población de soluciones; selección, cruza, mutación
	Colonia de hormigas	Construcciones guiadas por «feromonas» que se refuerzan
	Scatter search	Combinar soluciones élite recorriendo caminos entre ellas

Un zoológico

Familia	Metaheurística	Idea en una línea
Trayectoria	Multi-start / **GRASP**	Muchas construcciones greedy randomizadas + búsqueda local
	Simulated annealing	Aceptar empeoramientos con probabilidad $e^{-\Delta/T}$, T decreciente
	Búsqueda tabú	Siempre al mejor vecino, aunque empeore; prohibir deshacerlo
	Iterated local search	Óptimo local $\rightarrow$ perturbar $\rightarrow$ búsqueda local $\rightarrow$ aceptar o no
	VNS	Cambiar de vecindad cuando la actual se agota
Población	Algoritmos genéticos	Población de soluciones; selección, cruza, mutación
	Colonia de hormigas	Construcciones guiadas por «feromonas» que se refuerzan
	Scatter search	Combinar soluciones élite recorriendo caminos entre ellas

Vemos una en detalle, GRASP, porque es la más simple de explicar, implementar y evaluar; y dos más, en una línea cada una.

GRASP (Feo y Resende, 1989)

Greedy **R**andomized **A**daptive **S**earch **P**rocedure. Multi-start con dos fases por iteración:

---

function GRASP(*maxIter*, α)
    mejor $\leftarrow \perp$;
    for *k* $\leftarrow$ 1 **to** *maxIter* **do**
        s $\leftarrow$ construccionGreedyRandomizada(α) ; `// fase 1`
        s $\leftarrow$ busquedaLocal(*s*) ; `// fase 2`
        if *c*(*s*) $<$ *c*(*mejor*) **then** *mejor* $\leftarrow$ *s*;
    return *mejor*;

GRASP (Feo y Resende, 1989)

Greedy **R**andomized **A**daptive **S**earch **P**rocedure. Multi-start con dos fases por iteración:

```
function GRASP(maxIter, α)
    mejor ← ⊥;
    for k ← 1 to maxIter do
        s ← construccionGreedyRandomizada(α) ;        // fase 1
        s ← busquedaLocal(s) ;                        // fase 2
        if c(s) < c(mejor) then mejor ← s;
    return mejor;
```

- **Greedy** randomizado: un greedy que no siempre elige al mejor candidato. **Adaptive:** la función greedy se recalcula según lo construido. **Search:** búsqueda local después.

GRASP (Feo y Resende, 1989)

Greedy **R**andomized **A**daptive **S**earch **P**rocedure. Multi-start con dos fases por iteración:

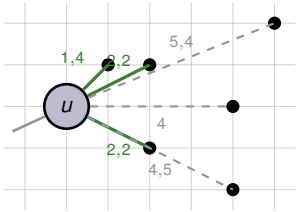
```
function GRASP(maxIter, α)
    mejor ← ⊥;
    for k ← 1 to maxIter do
        s ← construccionGreedyRandomizada(α) ;        // fase 1
        s ← busquedaLocal(s) ;                        // fase 2
        if c(s) < c(mejor) then mejor ← s;
    return mejor;
```

- **Greedy** randomizado: un greedy que no siempre elige al mejor candidato. **Adaptive:** la función greedy se recalcula según lo construido. **Search:** búsqueda local después.
- Iteraciones **independientes**: trivial de paralelizar; se para por iteraciones, tiempo o «k sin mejorar».

Fase 1: construcción greedy randomizada

En cada paso de la construcción:

1. $\mathcal{C}$ = candidatos (lo que se puede agregar); $g(c)$ = costo greedy de cada uno; $g_{\text{mín}}$, $g_{\text{máx}}$ el menor y el mayor.



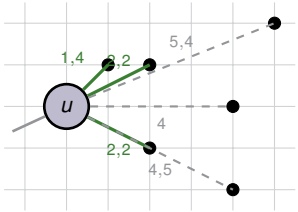
Vecino más cercano randomizado: desde u, $g_{\text{mín}} = 1{,}4$, $g_{\text{máx}} = 5{,}4$. Con $\alpha = 0{,}25$ el umbral es $1{,}4 + 0{,}25 \cdot 4 = 2{,}4$: la RCL tiene 3 ciudades (verdes). El greedy puro tomaría siempre la de 1,4.

Fase 1: construcción greedy randomizada

En cada paso de la construcción:

1. $\mathcal{C}$ = candidatos (lo que se puede agregar); $g(c)$ = costo greedy de cada uno; $g_{\text{mín}}$, $g_{\text{máx}}$ el menor y el mayor.
2. **Lista restringida de candidatos:**

$$\text{RCL} = \{c \in \mathcal{C} : g(c) \leq g_{\text{mín}} + \alpha\,(g_{\text{máx}} - g_{\text{mín}})\}.$$



Vecino más cercano randomizado: desde u, $g_{\text{mín}} = 1{,}4$, $g_{\text{máx}} = 5{,}4$. Con $\alpha = 0{,}25$ el umbral es $1{,}4 + 0{,}25 \cdot 4 = 2{,}4$: la RCL tiene 3 ciudades (verdes). El greedy puro tomaría siempre la de 1,4.

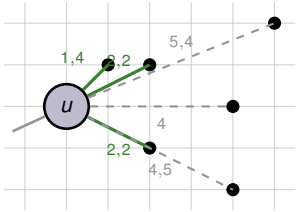
Fase 1: construcción greedy randomizada

En cada paso de la construcción:

1. $\mathcal{C}$ = candidatos (lo que se puede agregar); $g(c)$ = costo greedy de cada uno; $g_{\text{mín}}$, $g_{\text{máx}}$ el menor y el mayor.
2. **Lista restringida de candidatos:**

$$\text{RCL} = \{c \in \mathcal{C} : g(c) \leq g_{\text{mín}} + \alpha\,(g_{\text{máx}} - g_{\text{mín}})\}.$$

3. Elegir c **uniformemente al azar** en la RCL; agregarlo; actualizar $\mathcal{C}$ y g.



Vecino más cercano randomizado: desde u, $g_{\text{mín}} = 1{,}4$, $g_{\text{máx}} = 5{,}4$. Con $\alpha = 0{,}25$ el umbral es $1{,}4 + 0{,}25 \cdot 4 = 2{,}4$: la RCL tiene 3 ciudades (verdes). El greedy puro tomaría siempre la de 1,4.

Fase 1: construcción greedy randomizada

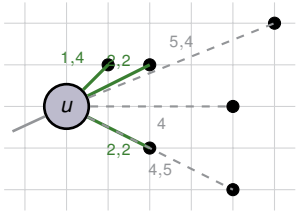
En cada paso de la construcción:

1. $\mathcal{C}$ = candidatos (lo que se puede agregar); $g(c)$ = costo greedy de cada uno; $g_{\text{mín}}$, $g_{\text{máx}}$ el menor y el mayor.
2. **Lista restringida de candidatos:**

$$\text{RCL} = \{c \in \mathcal{C} : g(c) \leq g_{\text{mín}} + \alpha\,(g_{\text{máx}} - g_{\text{mín}})\}.$$

3. Elegir c **uniformemente al azar** en la RCL; agregarlo; actualizar $\mathcal{C}$ y g.

- $\alpha = 0$: greedy puro, **determinista**: todas las iteraciones construyen lo mismo. $\alpha = 1$: **al azar**, sin criterio.



Vecino más cercano randomizado: desde u, $g_{\text{mín}} = 1{,}4$, $g_{\text{máx}} = 5{,}4$. Con $\alpha = 0{,}25$ el umbral es $1{,}4 + 0{,}25 \cdot 4 = 2{,}4$: la RCL tiene 3 ciudades (verdes). El greedy puro tomaría siempre la de 1,4.

Fase 2: búsqueda local, y por qué las dos juntas

- La construcción devuelve una solución *razonable* con defectos locales (como vecino más cercano: los últimos pasos son malos). La búsqueda local los arregla barato: arranca cerca de un óptimo local.

Fase 2: búsqueda local, y por qué las dos juntas

- La construcción devuelve una solución *razonable* con defectos locales (como vecino más cercano: los últimos pasos son malos). La búsqueda local los arregla barato: arranca cerca de un óptimo local.
- Construcción distinta en cada iteración $\Rightarrow$ la búsqueda local arranca en otro lado $\Rightarrow$ termina en un óptimo local *distinto*. GRASP **muestrea óptimos locales** y se queda con el mejor.

Fase 2: búsqueda local, y por qué las dos juntas

- La construcción devuelve una solución *razonable* con defectos locales (como vecino más cercano: los últimos pasos son malos). La búsqueda local los arregla barato: arranca cerca de un óptimo local.
- Construcción distinta en cada iteración $\Rightarrow$ la búsqueda local arranca en otro lado $\Rightarrow$ termina en un óptimo local *distinto*. GRASP **muestrea óptimos locales** y se queda con el mejor.
- Contra reinicios puramente aleatorios: mejores soluciones iniciales, búsqueda local más corta, óptimos locales mejores. Contra greedy puro: hay más de una solución para elegir.

Fase 2: búsqueda local, y por qué las dos juntas

- La construcción devuelve una solución *razonable* con defectos locales (como vecino más cercano: los últimos pasos son malos). La búsqueda local los arregla barato: arranca cerca de un óptimo local.
- Construcción distinta en cada iteración $\Rightarrow$ la búsqueda local arranca en otro lado $\Rightarrow$ termina en un óptimo local *distinto*. GRASP **muestrea óptimos locales** y se queda con el mejor.
- Contra reinicios puramente aleatorios: mejores soluciones iniciales, búsqueda local más corta, óptimos locales mejores. Contra greedy puro: hay más de una solución para elegir.
- α regula el compromiso: chico $=$ iniciales buenas pero parecidas (intensificación); grande $=$ diversas pero malas (diversificación).

Fase 2: búsqueda local, y por qué las dos juntas

- La construcción devuelve una solución *razonable* con defectos locales (como vecino más cercano: los últimos pasos son malos). La búsqueda local los arregla barato: arranca cerca de un óptimo local.
- Construcción distinta en cada iteración $\Rightarrow$ la búsqueda local arranca en otro lado $\Rightarrow$ termina en un óptimo local *distinto*. GRASP **muestrea óptimos locales** y se queda con el mejor.
- Contra reinicios puramente aleatorios: mejores soluciones iniciales, búsqueda local más corta, óptimos locales mejores. Contra greedy puro: hay más de una solución para elegir.
- α regula el compromiso: chico $=$ iniciales buenas pero parecidas (intensificación); grande $=$ diversas pero malas (diversificación).

¿Cuánto vale α? Depende del problema, la instancia y cuántas iteraciones podemos pagar. **Se calibra experimentalmente.** Veamos uno.

El azar en las heurísticas: para qué sirve

En las heurísticas el azar se usa para **diversificar**: que corridas distintas recorran lugares distintos del espacio de soluciones.

El azar en las heurísticas: para qué sirve

En las heurísticas el azar se usa para **diversificar**: que corridas distintas recorran lugares distintos del espacio de soluciones.

- **Reinicios al azar:** búsqueda local desde k iniciales aleatorias, quedarse con la mejor. Si desde una inicial al azar se llega al óptimo con probabilidad p, con k reinicios se lo alcanza con probabilidad $1 - (1 - p)^k$. (Pero no conocemos p, y puede ser 2^{-n}.)

El azar en las heurísticas: para qué sirve

En las heurísticas el azar se usa para **diversificar**: que corridas distintas recorran lugares distintos del espacio de soluciones.

- **Reinicios al azar:** búsqueda local desde k iniciales aleatorias, quedarse con la mejor. Si desde una inicial al azar se llega al óptimo con probabilidad p, con k reinicios se lo alcanza con probabilidad $1 - (1 - p)^k$. (Pero no conocemos p, y puede ser 2^{-n}.)
- **Desempates y vecinos al azar:** para no recorrer siempre el mismo camino.

El azar en las heurísticas: para qué sirve

En las heurísticas el azar se usa para **diversificar**: que corridas distintas recorran lugares distintos del espacio de soluciones.

- **Reinicios al azar:** búsqueda local desde k iniciales aleatorias, quedarse con la mejor. Si desde una inicial al azar se llega al óptimo con probabilidad p, con k reinicios se lo alcanza con probabilidad $1 - (1 - p)^k$. (Pero no conocemos p, y puede ser 2^{-n}.)
- **Desempates y vecinos al azar:** para no recorrer siempre el mismo camino.
- **Perturbaciones:** un movimiento grande al azar sobre un óptimo local, y buscar de nuevo (*iterated local search*).

El azar en las heurísticas: para qué sirve

En las heurísticas el azar se usa para **diversificar**: que corridas distintas recorran lugares distintos del espacio de soluciones.

- **Reinicios al azar:** búsqueda local desde k iniciales aleatorias, quedarse con la mejor. Si desde una inicial al azar se llega al óptimo con probabilidad p, con k reinicios se lo alcanza con probabilidad $1 - (1 - p)^k$. (Pero no conocemos p, y puede ser 2^{-n}.)
- **Desempates y vecinos al azar:** para no recorrer siempre el mismo camino.
- **Perturbaciones:** un movimiento grande al azar sobre un óptimo local, y buscar de nuevo (*iterated local search*).
- **Construcción randomizada:** la RCL de GRASP: elegir al azar entre los mejores candidatos, no siempre al mejor.

El azar en las heurísticas: para qué sirve

En las heurísticas el azar se usa para **diversificar**: que corridas distintas recorran lugares distintos del espacio de soluciones.

- **Reinicios al azar:** búsqueda local desde k iniciales aleatorias, quedarse con la mejor. Si desde una inicial al azar se llega al óptimo con probabilidad p, con k reinicios se lo alcanza con probabilidad $1 - (1 - p)^k$. (Pero no conocemos p, y puede ser 2^{-n}.)
- **Desempates y vecinos al azar:** para no recorrer siempre el mismo camino.
- **Perturbaciones:** un movimiento grande al azar sobre un óptimo local, y buscar de nuevo (*iterated local search*).
- **Construcción randomizada:** la RCL de GRASP: elegir al azar entre los mejores candidatos, no siempre al mejor.

Consecuencia: con azar, **una corrida no dice nada**. Varias corridas con semillas distintas (media, desviación, mejor), y guardar las semillas para reproducir.

Un experimento: TSP, $n = 60$, 100 iteraciones

Puntos uniformes en un cuadrado; construcción = vecino más cercano randomizado desde una ciudad *fija*; búsqueda local = 2-opt. Referencia: mejor solución en 4 500 iteraciones, costo 604,5.

	Sólo construcción		Construcción + 2-opt		Gap del mejor	
α	mejor	media	mejor	media	sin 2-opt	con 2-opt
0 (greedy puro)	706	706	647,0	647	16,7 %	7,0 %
0,05	693	799	609,8	643	14,7 %	0,9 %
0,1	778	911	606,6	653	28,6 %	0,3 %
0,3	1207	1407	**604,5**	658	99,6 %	0,0 %
1 (al azar)	2537	2951	606,7	651	319,6 %	0,4 %

Un experimento: TSP, $n = 60$, 100 iteraciones

Puntos uniformes en un cuadrado; construcción = vecino más cercano randomizado desde una ciudad *fija*; búsqueda local = 2-opt. Referencia: mejor solución en 4 500 iteraciones, costo 604,5.

	Sólo construcción		Construcción + 2-opt		Gap del mejor	
α	mejor	media	mejor	media	sin 2-opt	con 2-opt
0 (greedy puro)	706	706	647,0	647	16,7 %	7,0 %
0,05	693	799	609,8	643	14,7 %	0,9 %
0,1	778	911	606,6	653	28,6 %	0,3 %
0,3	1207	1407	**604,5**	658	99,6 %	0,0 %
1 (al azar)	2537	2951	606,7	651	319,6 %	0,4 %

- Con $\alpha = 0$ las 100 iteraciones dan **la misma** solución. Cualquier $\alpha > 0$ la mejora.

Un experimento: TSP, $n = 60$, 100 iteraciones

Puntos uniformes en un cuadrado; construcción = vecino más cercano randomizado desde una ciudad *fija*; búsqueda local = 2-opt. Referencia: mejor solución en 4 500 iteraciones, costo 604,5.

	Sólo construcción		Construcción + 2-opt		Gap del mejor	
α	mejor	media	mejor	media	sin 2-opt	con 2-opt
0 (greedy puro)	706	706	647,0	647	16,7 %	7,0 %
0,05	693	799	609,8	643	14,7 %	0,9 %
0,1	778	911	606,6	653	28,6 %	0,3 %
0,3	1207	1407	**604,5**	658	99,6 %	0,0 %
1 (al azar)	2537	2951	606,7	651	319,6 %	0,4 %

- Con $\alpha = 0$ las 100 iteraciones dan **la misma** solución. Cualquier $\alpha > 0$ la mejora.
- La construcción sola se degrada rápido; 2-opt lo compensa. La *media* con 2-opt es parecida para todo α: lo que gana GRASP es el **mejor de muchos**.

Un experimento: TSP, $n = 60$, 100 iteraciones

Puntos uniformes en un cuadrado; construcción = vecino más cercano randomizado desde una ciudad *fija*; búsqueda local = 2-opt. Referencia: mejor solución en 4 500 iteraciones, costo 604,5.

	Sólo construcción		Construcción + 2-opt		Gap del mejor	
α	mejor	media	mejor	media	sin 2-opt	con 2-opt
0 (greedy puro)	706	706	647,0	647	16,7 %	7,0 %
0,05	693	799	609,8	643	14,7 %	0,9 %
0,1	778	911	606,6	653	28,6 %	0,3 %
0,3	1207	1407	**604,5**	658	99,6 %	0,0 %
1 (al azar)	2537	2951	606,7	651	319,6 %	0,4 %

- Con $\alpha = 0$ las 100 iteraciones dan **la misma** solución. Cualquier $\alpha > 0$ la mejora.
- La construcción sola se degrada rápido; 2-opt lo compensa. La *media* con 2-opt es parecida para todo α: lo que gana GRASP es el **mejor de muchos**.
- Con una instancia no se puede concluir «$\alpha = 0{,}3$ es el mejor»: diferencias < 1 %, hay ruido. Varias instancias, varias semillas. *Ese es el trabajo.*

Dos más que hay que conocer

Simulated annealing (Kirkpatrick, Gelatt y Vecchi, 1983; K–T 12.2)

Elegir un vecino s' al azar; aceptarlo *siempre* si mejora, y con probabilidad $e^{-(c(s')-c(s))/T}$ si empeora. La **temperatura** T empieza alta (casi paseo al azar) y baja de a poco (casi búsqueda local). Inspirado en el recocido de metales. Funciona muy bien con un programa de enfriamiento bien calibrado.

Dos más que hay que conocer

Simulated annealing (Kirkpatrick, Gelatt y Vecchi, 1983; K–T 12.2)

Elegir un vecino s' al azar; aceptarlo *siempre* si mejora, y con probabilidad $e^{-(c(s')-c(s))/T}$ si empeora. La **temperatura** T empieza alta (casi paseo al azar) y baja de a poco (casi búsqueda local). Inspirado en el recocido de metales. Funciona muy bien con un programa de enfriamiento bien calibrado.

Búsqueda tabú (Glover, 1986)

Ir *siempre* al mejor vecino, aunque sea peor que la solución actual (así se sale del óptimo local). Para no volver enseguida, una **lista tabú** prohíbe deshacer los últimos k movimientos. Determinista: la memoria reemplaza al azar.

Dos más que hay que conocer

Simulated annealing (Kirkpatrick, Gelatt y Vecchi, 1983; K–T 12.2)

Elegir un vecino s' al azar; aceptarlo *siempre* si mejora, y con probabilidad $e^{-(c(s')-c(s))/T}$ si empeora. La **temperatura** T empieza alta (casi paseo al azar) y baja de a poco (casi búsqueda local). Inspirado en el recocido de metales. Funciona muy bien con un programa de enfriamiento bien calibrado.

Búsqueda tabú (Glover, 1986)

Ir *siempre* al mejor vecino, aunque sea peor que la solución actual (así se sale del óptimo local). Para no volver enseguida, una **lista tabú** prohíbe deshacer los últimos k movimientos. Determinista: la memoria reemplaza al azar.

Se combinan (GRASP con tabú como fase 2, por ejemplo) y necesitan lo mismo: una buena vecindad, evaluación incremental y un protocolo experimental honesto.

Trampas comunes al evaluar una metaheurística

1. **Comparar a distinto presupuesto.** Cien iteraciones de GRASP contra una corrida de búsqueda local no es una comparación: hay que igualar tiempo.

Trampas comunes al evaluar una metaheurística

1. **Comparar a distinto presupuesto.** Cien iteraciones de GRASP contra una corrida de búsqueda local no es una comparación: hay que igualar tiempo.
2. **Calibrar y reportar sobre las mismas instancias.** Elegir α en las instancias del informe es *overfitting*.

Trampas comunes al evaluar una metaheurística

1. **Comparar a distinto presupuesto.** Cien iteraciones de GRASP contra una corrida de búsqueda local no es una comparación: hay que igualar tiempo.
2. **Calibrar y reportar sobre las mismas instancias.** Elegir α en las instancias del informe es *overfitting*.
3. **Una sola semilla.** El mejor resultado de una corrida es suerte. Media, desviación, mejor; y cuántas corridas.

Trampas comunes al evaluar una metaheurística

1. **Comparar a distinto presupuesto.** Cien iteraciones de GRASP contra una corrida de búsqueda local no es una comparación: hay que igualar tiempo.
2. **Calibrar y reportar sobre las mismas instancias.** Elegir α en las instancias del informe es *overfitting*.
3. **Una sola semilla.** El mejor resultado de una corrida es suerte. Media, desviación, mejor; y cuántas corridas.
4. **No verificar factibilidad.** Toda solución reportada se *verifica* con un chequeo independiente. Un bug en la evaluación incremental produce «mejoras» que no existen.

Trampas comunes al evaluar una metaheurística

1. **Comparar a distinto presupuesto.** Cien iteraciones de GRASP contra una corrida de búsqueda local no es una comparación: hay que igualar tiempo.
2. **Calibrar y reportar sobre las mismas instancias.** Elegir α en las instancias del informe es *overfitting*.
3. **Una sola semilla.** El mejor resultado de una corrida es suerte. Media, desviación, mejor; y cuántas corridas.
4. **No verificar factibilidad.** Toda solución reportada se *verifica* con un chequeo independiente. Un bug en la evaluación incremental produce «mejoras» que no existen.
5. **Sin baseline.** Si el greedy simple da lo mismo que GRASP, GRASP no aporta; hay que mostrarlo.

Trampas comunes al evaluar una metaheurística

1. **Comparar a distinto presupuesto.** Cien iteraciones de GRASP contra una corrida de búsqueda local no es una comparación: hay que igualar tiempo.
2. **Calibrar y reportar sobre las mismas instancias.** Elegir α en las instancias del informe es *overfitting*.
3. **Una sola semilla.** El mejor resultado de una corrida es suerte. Media, desviación, mejor; y cuántas corridas.
4. **No verificar factibilidad.** Toda solución reportada se *verifica* con un chequeo independiente. Un bug en la evaluación incremental produce «mejoras» que no existen.
5. **Sin baseline.** Si el greedy simple da lo mismo que GRASP, GRASP no aporta; hay que mostrarlo.
6. **Sin referencia.** Un costo de 604,5 no significa nada sin una cota inferior o un óptimo conocido: en instancias chicas, correr el algoritmo exacto.

Volvamos a camino mínimo



Dijkstra de s a t: expande 39 celdas de 45.

- Queremos el camino mínimo de s a t (un solo destino), con pesos ≥ 0.

Volvamos a camino mínimo



Dijkstra de s a t: expande 39 celdas de 45.

- Queremos el camino mínimo de s a t (un solo destino), con pesos ≥ 0.
- **Dijkstra** extrae los vértices en orden de $g(v) = \text{dist}(s, v)$: explora «en círculos» alrededor de s y se detiene recién al extraer t. Visita *todo* vértice con $\text{dist}(s, v) < \text{dist}(s, t)$, incluso los que están en la dirección opuesta.

Volvamos a camino mínimo



Dijkstra de s a t: expande 39 celdas de 45.

- Queremos el camino mínimo de s a t (un solo destino), con pesos ≥ 0.
- **Dijkstra** extrae los vértices en orden de $g(v) = \text{dist}(s, v)$: explora «en círculos» alrededor de s y se detiene recién al extraer t. Visita *todo* vértice con $\text{dist}(s, v) < \text{dist}(s, t)$, incluso los que están en la dirección opuesta.
- Pero sabemos *dónde* está t. ¿No podríamos usar eso?

Volvamos a camino mínimo



Dijkstra de s a t: expande 39 celdas de 45.

- Queremos el camino mínimo de s a t (un solo destino), con pesos ≥ 0.
- **Dijkstra** extrae los vértices en orden de $g(v) = \text{dist}(s, v)$: explora «en círculos» alrededor de s y se detiene recién al extraer t. Visita *todo* vértice con $\text{dist}(s, v) < \text{dist}(s, t)$, incluso los que están en la dirección opuesta.
- Pero sabemos *dónde* está t. ¿No podríamos usar eso?

Grilla de 9×5, movimientos a las 4 celdas adyacentes, costo 1. Celeste: extraídas de la cola (*expandidas*). Amarillo: el camino encontrado.

A* (Hart, Nilsson y Raphael, 1968)

La idea

Tener una **estimación** $h(v)$ de lo que falta, $\mathrm{dist}(v, t)$, y extraer en orden de $f(v) = g(v) + h(v)$: lo que costó llegar más lo que estimamos que falta. $f(v)$ estima el costo del mejor camino $s \to t$ *que pasa por* v.

A* (Hart, Nilsson y Raphael, 1968)

La idea

Tener una **estimación** $h(v)$ de lo que falta, $\mathrm{dist}(v, t)$, y extraer en orden de $f(v) = g(v) + h(v)$: lo que costó llegar más lo que estimamos que falta. $f(v)$ estima el costo del mejor camino $s \to t$ *que pasa por* v.

- h es una **función heurística**: la segunda acepción. La damos nosotros. Grilla: $h(v) = |x_v - x_t| + |y_v - y_t|$ (Manhattan). Mapa: distancia en línea recta. 8-puzzle: fichas fuera de lugar.

A* (Hart, Nilsson y Raphael, 1968)

La idea

Tener una **estimación** $h(v)$ de lo que falta, $\mathrm{dist}(v, t)$, y extraer en orden de $f(v) = g(v) + h(v)$: lo que costó llegar más lo que estimamos que falta. $f(v)$ estima el costo del mejor camino $s \to t$ *que pasa por* v.

- h es una **función heurística**: la segunda acepción. La damos nosotros. Grilla: $h(v) = |x_v - x_t| + |y_v - y_t|$ (Manhattan). Mapa: distancia en línea recta. 8-puzzle: fichas fuera de lugar.
- $h \equiv 0$: A* **es** Dijkstra. $h = h^*$ (la distancia real): expande sólo un camino óptimo. En el medio, lo interesante.

A* (Hart, Nilsson y Raphael, 1968)

La idea

Tener una **estimación** $h(v)$ de lo que falta, $\mathrm{dist}(v, t)$, y extraer en orden de $f(v) = g(v) + h(v)$: lo que costó llegar más lo que estimamos que falta. $f(v)$ estima el costo del mejor camino $s \to t$ *que pasa por* v.

- h es una **función heurística**: la segunda acepción. La damos nosotros. Grilla: $h(v) = |x_v - x_t| + |y_v - y_t|$ (Manhattan). Mapa: distancia en línea recta. 8-puzzle: fichas fuera de lugar.
- $h \equiv 0$: A* **es** Dijkstra. $h = h^*$ (la distancia real): expande sólo un camino óptimo. En el medio, lo interesante.
- Lo que hay que entender: qué condiciones sobre h hacen que A* **siga encontrando el óptimo**. Si h es cualquier cosa, no.

Pseudocódigo

function AEstrella($G,\ w,\ s,\ t,\ h$)
    $g(s) \leftarrow 0$; $g(v) \leftarrow \infty$ si $v \neq s$; $Q \leftarrow \{s\}$; *cerrados* $\leftarrow \emptyset$;
    while $Q \neq \emptyset$ **do**
        $u \leftarrow$ extraer de Q el de menor $f(u) = g(u) + h(u)$; agregar u a *cerrados*;
        if $u = t$ **then return** $g(t)$ (el camino, por los padres π);
        foreach *arista* (u, v) *con* $v \notin$ *cerrados y* $g(u) + w(u, v) < g(v)$ **do**
            $g(v) \leftarrow g(u) + w(u, v)$; $\pi(v) \leftarrow u$; insertar o actualizar v en Q con prioridad $g(v) + h(v)$;

Pseudocódigo

function AEstrella(G, w, s, t, h)
  $g(s) \leftarrow 0$; $g(v) \leftarrow \infty$ si $v \neq s$; $Q \leftarrow \{s\}$; *cerrados* $\leftarrow \emptyset$;
  while $Q \neq \emptyset$ **do**
    $u \leftarrow$ extraer de Q el de menor $f(u) = g(u) + h(u)$; agregar u a *cerrados*;
    if $u = t$ **then return** $g(t)$ (el camino, por los padres π);
    foreach *arista* (u, v) *con* $v \notin$ *cerrados y* $g(u) + w(u, v) < g(v)$ **do**
      $g(v) \leftarrow g(u) + w(u, v)$; $\pi(v) \leftarrow u$; insertar o actualizar v en Q con prioridad $g(v) + h(v)$;

Es Dijkstra con **dos cambios**: prioridad $g + h$ en vez de g, y corte al extraer t; los vértices extraídos (*cerrados*) no se vuelven a tocar. Mismo peor caso, $O((n + m) \log n)$ con un heap: la ganancia está en *cuántos* vértices se extraen antes que t.

Qué le pedimos a h

Definiciones

Sea $h^*(v) = \mathrm{dist}(v, t)$ la distancia real. Una heurística $h : V \to \mathbb{R}_{\geq 0}$ es

- **admisible** si $h(v) \leq h^*(v)$ para todo v: *nunca sobreestima* lo que falta;
- **consistente** (o *monótona*) si $h(t) = 0$ y $h(u) \leq w(u, v) + h(v)$ para toda arista (u, v): una desigualdad triangular.

Qué le pedimos a h

Definiciones

Sea $h^*(v) = \text{dist}(v, t)$ la distancia real. Una heurística $h : V \to \mathbb{R}_{\geq 0}$ es

- **admisible** si $h(v) \leq h^*(v)$ para todo v: *nunca sobreestima* lo que falta;
- **consistente** (o *monótona*) si $h(t) = 0$ y $h(u) \leq w(u, v) + h(v)$ para toda arista (u, v): una desigualdad triangular.

- **Lema:** consistente $\Rightarrow$ admisible. Inducción en la cantidad de aristas de un camino mínimo de v a t. $\longrightarrow$ Pizarrón.

Qué le pedimos a *h*

Definiciones

Sea $h^*(v) = \mathrm{dist}(v, t)$ la distancia real. Una heurística $h : V \to \mathbb{R}_{\geq 0}$ es

- **admisible** si $h(v) \leq h^*(v)$ para todo v: *nunca sobreestima* lo que falta;
- **consistente** (o *monótona*) si $h(t) = 0$ y $h(u) \leq w(u, v) + h(v)$ para toda arista (u, v): una desigualdad triangular.

- **Lema:** consistente $\Rightarrow$ admisible. Inducción en la cantidad de aristas de un camino mínimo de v a t. $\longrightarrow$ Pizarrón.
- Las heurísticas «geométricas» (Manhattan sin muros, línea recta) son consistentes: cumplen la desigualdad triangular y son $\leq$ que cualquier camino. Es lo usual y lo que hay que pedir.

Qué le pedimos a *h*

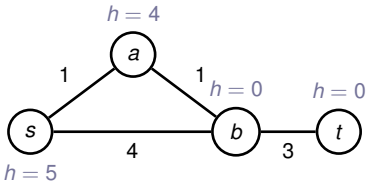
Definiciones

Sea $h^*(v) = \mathrm{dist}(v, t)$ la distancia real. Una heurística $h : V \to \mathbb{R}_{\geq 0}$ es

- **admisible** si $h(v) \leq h^*(v)$ para todo v: *nunca sobreestima* lo que falta;
- **consistente** (o *monótona*) si $h(t) = 0$ y $h(u) \leq w(u, v) + h(v)$ para toda arista (u, v): una desigualdad triangular.

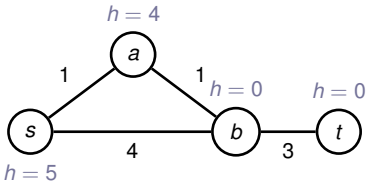
- **Lema:** consistente $\Rightarrow$ admisible. Inducción en la cantidad de aristas de un camino mínimo de v a t. $\longrightarrow$ Pizarrón.
- Las heurísticas «geométricas» (Manhattan sin muros, línea recta) son consistentes: cumplen la desigualdad triangular y son $\leq$ que cualquier camino. Es lo usual y lo que hay que pedir.
- La recíproca del lema es falsa, y la diferencia **importa**:

Admisible pero no consistente: qué se rompe



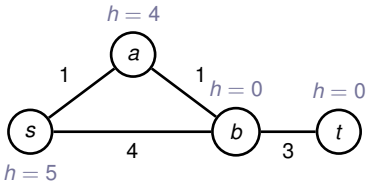
- h es admisible: $h(a) = 4 = h^*(a)$, $h(b) = 0 \leq 3$, $h(s) = 5 = h^*(s)$.

Admisible pero no consistente: qué se rompe



- h es admisible: $h(a) = 4 = h^*(a)$, $h(b) = 0 \leq 3$, $h(s) = 5 = h^*(s)$.
- No es consistente: $h(a) = 4 > w(a, b) + h(b) = 1$.

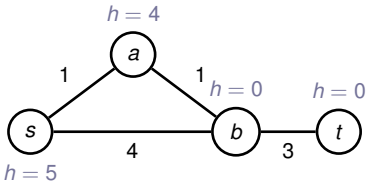
Admisible pero no consistente: qué se rompe



- h es admisible: $h(a) = 4 = h^*(a)$, $h(b) = 0 \leq 3$, $h(s) = 5 = h^*(s)$.
- No es consistente: $h(a) = 4 > w(a, b) + h(b) = 1$.

- A* extrae s e inserta a con $f = 1 + 4 = 5$ y b con $f = 4 + 0 = 4$. Extrae b (con $g(b) = 4$) e inserta t con $f = 7$.

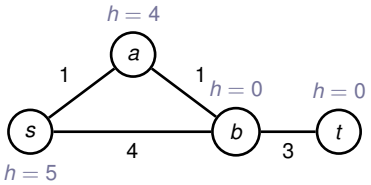
Admisible pero no consistente: qué se rompe



- h es admisible: $h(a) = 4 = h^*(a)$, $h(b) = 0 \leq 3$, $h(s) = 5 = h^*(s)$.
- No es consistente: $h(a) = 4 > w(a, b) + h(b) = 1$.

- A* extrae s e inserta a con $f = 1 + 4 = 5$ y b con $f = 4 + 0 = 4$. Extrae b (con $g(b) = 4$) e inserta t con $f = 7$.
- Extrae a ($f = 5$) y descubre $g(b) = 2 \ldots$ pero b ya fue extraído. Si no lo *reabrimos*, devuelve 7. El óptimo es 5.

Admisible pero no consistente: qué se rompe



- h es admisible: $h(a) = 4 = h^*(a)$, $h(b) = 0 \leq 3$, $h(s) = 5 = h^*(s)$.
- No es consistente: $h(a) = 4 > w(a, b) + h(b) = 1$.

- A* extrae s e inserta a con $f = 1 + 4 = 5$ y b con $f = 4 + 0 = 4$. Extrae b (con $g(b) = 4$) e inserta t con $f = 7$.
- Extrae a ($f = 5$) y descubre $g(b) = 2 \ldots$ pero b ya fue extraído. Si no lo *reabrimos*, devuelve 7. El óptimo es 5.
- **Con h sólo admisible**, A* es óptimo si se permite reabrir vértices (o se lo corre como búsqueda en árbol), y puede reabrir muchas veces. **Con h consistente**, nunca hace falta:

Teorema: con h consistente, A* es correcto

Teorema. Si $w \geq 0$ y h es consistente, A* devuelve $\mathrm{dist}(s, t)$ y extrae cada vértice a lo sumo una vez.

Demostración por repesado (el truco del algoritmo de Johnson):

Teorema: con h consistente, A* es correcto

Teorema. Si $w \geq 0$ y h es consistente, A* devuelve $\mathrm{dist}(s, t)$ y extrae cada vértice a lo sumo una vez.

Demostración por repesado (el truco del algoritmo de Johnson):

- Pesos nuevos $w'(u, v) = w(u, v) - h(u) + h(v)$. Por consistencia, $w'(u, v) \geq 0$.

Teorema: con h consistente, A* es correcto

Teorema. Si $w \geq 0$ y h es consistente, A* devuelve $\mathrm{dist}(s, t)$ y extrae cada vértice a lo sumo una vez.

Demostración por repesado (el truco del algoritmo de Johnson):

- Pesos nuevos $w'(u, v) = w(u, v) - h(u) + h(v)$. Por consistencia, $w'(u, v) \geq 0$.
- Para *cualquier* camino P de s a t la suma telescopa: $w'(P) = w(P) - h(s) + h(t) = w(P) - h(s)$. Todos los caminos $s \to t$ se corren la **misma constante**: los mínimos de (G, w') son los de (G, w).

Teorema: con h consistente, A* es correcto

Teorema. Si $w \geq 0$ y h es consistente, A* devuelve $\mathrm{dist}(s, t)$ y extrae cada vértice a lo sumo una vez.

Demostración por repesado (el truco del algoritmo de Johnson):

- Pesos nuevos $w'(u, v) = w(u, v) - h(u) + h(v)$. Por consistencia, $w'(u, v) \geq 0$.
- Para *cualquier* camino P de s a t la suma telescopa: $w'(P) = w(P) - h(s) + h(t) = w(P) - h(s)$. Todos los caminos $s \to t$ se corren la **misma constante**: los mínimos de (G, w') son los de (G, w).
- Dijkstra sobre (G, w') extrae por $g'(v) = g(v) - h(s) + h(v) = f(v) - h(s)$: **el mismo orden** que A* con f. Las relajaciones también coinciden.

Teorema: con h consistente, A* es correcto

Teorema. Si $w \geq 0$ y h es consistente, A* devuelve $\mathrm{dist}(s, t)$ y extrae cada vértice a lo sumo una vez.

Demostración por repesado (el truco del algoritmo de Johnson):

- Pesos nuevos $w'(u, v) = w(u, v) - h(u) + h(v)$. Por consistencia, $w'(u, v) \geq 0$.
- Para *cualquier* camino P de s a t la suma telescopa: $w'(P) = w(P) - h(s) + h(t) = w(P) - h(s)$. Todos los caminos $s \to t$ se corren la **misma constante**: los mínimos de (G, w') son los de (G, w).
- Dijkstra sobre (G, w') extrae por $g'(v) = g(v) - h(s) + h(v) = f(v) - h(s)$: **el mismo orden** que A* con f. Las relajaciones también coinciden.
- Luego A* sobre (G, w, h) *es* Dijkstra sobre (G, w') con $w' \geq 0$: hereda su correctitud (clase 10) y que cada vértice se extrae una sola vez. $\longrightarrow$ Pizarrón.

Teorema: con h consistente, A* es correcto

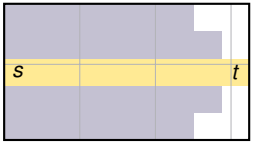
Teorema. Si $w \geq 0$ y h es consistente, A* devuelve $\mathrm{dist}(s, t)$ y extrae cada vértice a lo sumo una vez.

Demostración por repesado (el truco del algoritmo de Johnson):

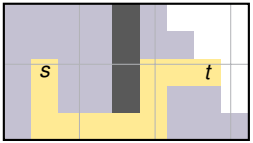
- Pesos nuevos $w'(u, v) = w(u, v) - h(u) + h(v)$. Por consistencia, $w'(u, v) \geq 0$.
- Para *cualquier* camino P de s a t la suma telescopa: $w'(P) = w(P) - h(s) + h(t) = w(P) - h(s)$. Todos los caminos $s \to t$ se corren la **misma constante**: los mínimos de (G, w') son los de (G, w).
- Dijkstra sobre (G, w') extrae por $g'(v) = g(v) - h(s) + h(v) = f(v) - h(s)$: **el mismo orden** que A* con f. Las relajaciones también coinciden.
- Luego A* sobre (G, w, h) *es* Dijkstra sobre (G, w') con $w' \geq 0$: hereda su correctitud (clase 10) y que cada vértice se extrae una sola vez. $\longrightarrow$ Pizarrón.

La consistencia es *exactamente* $w' \geq 0$: la hipótesis que Dijkstra necesita. Nada más y nada menos.

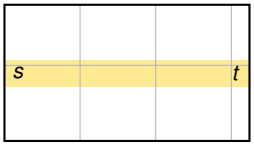
Dijkstra vs. A* en la grilla



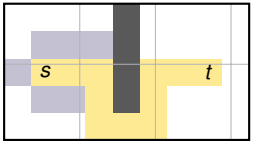
Dijkstra ($h \equiv 0$): 39 celdas.



Con un muro: 34 celdas.

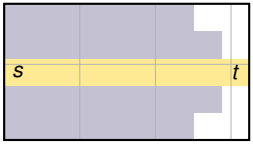


A* con Manhattan: 9 celdas (sólo el camino).

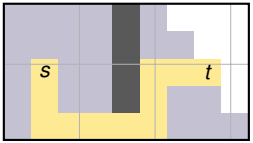


Con un muro: 17 celdas. Mismo óptimo (10).

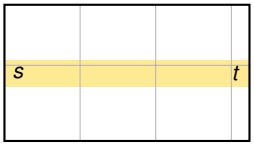
Dijkstra vs. A* en la grilla



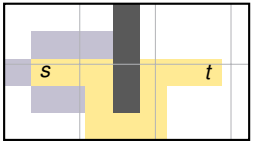
Dijkstra ($h \equiv 0$): 39 celdas.



Con un muro: 34 celdas.



A* con Manhattan: 9 celdas (sólo el camino).



Con un muro: 17 celdas. Mismo óptimo (10).

Sin obstáculos, las celdas entre s y t tienen $f = 8$ y A* expande sólo las de la dirección correcta (desempata por mayor g). Con el muro, Manhattan «miente» y expande más. En ambos casos el camino es óptimo: el teorema no pide que h sea buena, sólo consistente.

¿De dónde salen las heurísticas admisibles?

La receta: relajar. Quitar restricciones. El óptimo del problema relajado es $\leq$ el del real (hay más soluciones): una **cota inferior**, o sea, una heurística admisible; si es una distancia, consistente.

Problema	Restricción que se quita	Heurística
Grilla con obstáculos	«No atravesar muros»	Distancia Manhattan
8-puzzle	«Sólo se mueve al hueco»	Suma de Manhattan de cada ficha
TSP (como búsqueda)	«El resto tiene que ser un camino»	Árbol generador mínimo de lo que falta

¿De dónde salen las heurísticas admisibles?

La receta: relajar. Quitar restricciones. El óptimo del problema relajado es $\leq$ el del real (hay más soluciones): una **cota inferior**, o sea, una heurística admisible; si es una distancia, consistente.

Problema	Restricción que se quita	Heurística
Grilla con obstáculos	«No atravesar muros»	Distancia Manhattan
8-puzzle	«Sólo se mueve al hueco»	Suma de Manhattan de cada ficha
TSP (como búsqueda)	«El resto tiene que ser un camino»	Árbol generador mínimo de lo que falta

- Es lo que hicimos con las cotas de branch and bound (clase 5). A* es branch and bound *best-first* para camino mínimo.

¿De dónde salen las heurísticas admisibles?

La receta: relajar. Quitar restricciones. El óptimo del problema relajado es $\leq$ el del real (hay más soluciones): una **cota inferior**, o sea, una heurística admisible; si es una distancia, consistente.

Problema	Restricción que se quita	Heurística
Grilla con obstáculos	«No atravesar muros»	Distancia Manhattan
8-puzzle	«Sólo se mueve al hueco»	Suma de Manhattan de cada ficha
TSP (como búsqueda)	«El resto tiene que ser un camino»	Árbol generador mínimo de lo que falta

- Es lo que hicimos con las cotas de branch and bound (clase 5). A* es branch and bound *best-first* para camino mínimo.
- **Más informada es mejor:** si $h_1 \leq h_2 \leq h^*$ (consistentes), con h_2 se expande un subconjunto de lo que se expande con h_1 (salvo empates). El límite: el costo de calcular h.

Todo lo de hoy, en una línea de código

Cambiando sólo la prioridad con que se extrae de la cola:

Prioridad	Nombre	¿Qué garantiza?	¿Qué es?
$g(v)$	Dijkstra	El óptimo	Exacto
$g(v) + h(v)$, h consistente	A*	El óptimo, explorando menos	Exacto
$g(v) + \lambda\, h(v)$, $\lambda > 1$, h admisible	A* pesado (Pohl, 1970)	Costo $\leq \lambda \cdot \mathrm{OPT}$	**Aproximación** ($\rho = \lambda$)
$h(v)$	Greedy best-first	Nada	**Heurística**
$g(v) + h(v)$, h no admisible	—	Nada	**Heurística**

Todo lo de hoy, en una línea de código

Cambiando sólo la prioridad con que se extrae de la cola:

Prioridad	Nombre	¿Qué garantiza?	¿Qué es?
$g(v)$	Dijkstra	El óptimo	Exacto
$g(v) + h(v)$, h consistente	A*	El óptimo, explorando menos	Exacto
$g(v) + \lambda\, h(v)$, $\lambda > 1$, h admisible	A* pesado (Pohl, 1970)	Costo $\leq \lambda \cdot \mathrm{OPT}$	**Aproximación** ($\rho = \lambda$)
$h(v)$	Greedy best-first	Nada	**Heurística**
$g(v) + h(v)$, h no admisible	—	Nada	**Heurística**

- A* pesado es un dial entre **tiempo** y **garantía**, y la garantía está *demostrada* para cada λ (pizarrón, opcional).

Todo lo de hoy, en una línea de código

Cambiando sólo la prioridad con que se extrae de la cola:

Prioridad	Nombre	¿Qué garantiza?	¿Qué es?
$g(v)$	Dijkstra	El óptimo	Exacto
$g(v) + h(v)$, h consistente	A*	El óptimo, explorando menos	Exacto
$g(v) + \lambda\, h(v)$, $\lambda > 1$, h admisible	A* pesado (Pohl, 1970)	Costo $\leq \lambda \cdot \mathrm{OPT}$	**Aproximación** ($\rho = \lambda$)
$h(v)$	Greedy best-first	Nada	**Heurística**
$g(v) + h(v)$, h no admisible	—	Nada	**Heurística**

- A* pesado es un dial entre **tiempo** y **garantía**, y la garantía está *demostrada* para cada λ (pizarrón, opcional).
- En la práctica (GPS, videojuegos, planificación): heurísticas consistentes precalculadas (*landmarks*). El cuello de botella suele ser la **memoria** de la cola.

El glosario de hoy, completo

Término	¿Qué es? ¿Qué garantiza?	Ejemplo canónico
Búsqueda local	Moverse a una vecina mejor hasta que no haya. Garantiza un **óptimo local**; nada sobre OPT.	2-opt para TSP; min-conflicts
Heurística	Algoritmo que busca una solución buena, rápido, **sin garantía** de calidad. Se evalúa con experimentos.	Vecino más cercano
Algoritmo de aproximación	Polinomial, con **demostración** de $\mathrm{ALG} \leq \rho \cdot \mathrm{OPT}$ en *toda* instancia.	Matching maximal (VC, $\rho = 2$)
Metaheurística	Estrategia general, independiente del problema, para guiar heurísticas más allá de un óptimo local. Sin garantía.	GRASP (construcción greedy randomizada + búsqueda local)
A*	Dijkstra con prioridad $g+h$. Con h consistente, **devuelve el óptimo** explorando menos.	Grilla con Manhattan

Los pares que se confunden

1. **Heurística vs. aproximación.** Las dos ceden optimalidad. La aproximación tiene un *teorema* de peor caso; la heurística, *experimentos*. Un mismo algoritmo puede ser las dos (búsqueda local para Max-Cut).

Los pares que se confunden

1. **Heurística vs. aproximación.** Las dos ceden optimalidad. La aproximación tiene un *teorema* de peor caso; la heurística, *experimentos*. Un mismo algoritmo puede ser las dos (búsqueda local para Max-Cut).
2. **Heurística (algoritmo) vs. heurística (función).** Un algoritmo sin garantía, o una estimación h que guía a un algoritmo exacto. A* usa la segunda y no cede nada.

Los pares que se confunden

1. **Heurística vs. aproximación.** Las dos ceden optimalidad. La aproximación tiene un *teorema* de peor caso; la heurística, *experimentos*. Un mismo algoritmo puede ser las dos (búsqueda local para Max-Cut).
2. **Heurística (algoritmo) vs. heurística (función).** Un algoritmo sin garantía, o una estimación h que guía a un algoritmo exacto. A* usa la segunda y no cede nada.
3. **Admisible vs. consistente.** $h \leq h^*$, o $h(u) \leq w(u, v) + h(v)$ con $h(t) = 0$. Consistente $\Rightarrow$ admisible, no al revés. Con consistente, A* es Dijkstra repesado y no reabre.

Los pares que se confunden

1. **Heurística vs. aproximación.** Las dos ceden optimalidad. La aproximación tiene un *teorema* de peor caso; la heurística, *experimentos*. Un mismo algoritmo puede ser las dos (búsqueda local para Max-Cut).
2. **Heurística (algoritmo) vs. heurística (función).** Un algoritmo sin garantía, o una estimación h que guía a un algoritmo exacto. A* usa la segunda y no cede nada.
3. **Admisible vs. consistente.** $h \leq h^*$, o $h(u) \leq w(u, v) + h(v)$ con $h(t) = 0$. Consistente $\Rightarrow$ admisible, no al revés. Con consistente, A* es Dijkstra repesado y no reabre.
4. **Óptimo local vs. global.** Depende de la vecindad: óptimo local para 2-opt puede no serlo para 3-opt. «Óptimo local» sin decir de qué vecindad no significa nada.

Preguntas que podrían aparecer en el parcial (I)

1. Definí *vecindad* y *óptimo local*. ¿Por qué la búsqueda local termina? ¿Qué garantiza?

Preguntas que podrían aparecer en el parcial (I)

1. Definí *vecindad* y *óptimo local*. ¿Por qué la búsqueda local termina? ¿Qué garantiza?
2. ¿Qué diferencia hay entre una heurística y un algoritmo de aproximación? Da un ejemplo de cada uno para el mismo problema.

Preguntas que podrían aparecer en el parcial (I)

1. Definí *vecindad* y *óptimo local*. ¿Por qué la búsqueda local termina? ¿Qué garantiza?
2. ¿Qué diferencia hay entre una heurística y un algoritmo de aproximación? Da un ejemplo de cada uno para el mismo problema.
3. Enunciá el algoritmo de matching maximal para vertex cover y demostrá que es una 2-aproximación. ¿Es ajustada la cota?

Preguntas que podrían aparecer en el parcial (I)

1. Definí *vecindad* y *óptimo local*. ¿Por qué la búsqueda local termina? ¿Qué garantiza?
2. ¿Qué diferencia hay entre una heurística y un algoritmo de aproximación? Da un ejemplo de cada uno para el mismo problema.
3. Enunciá el algoritmo de matching maximal para vertex cover y demostrá que es una 2-aproximación. ¿Es ajustada la cota?
4. Enunciá el algoritmo del árbol generador mínimo para TSP métrico y demostrá su factor. ¿Dónde se usa la desigualdad triangular? ¿Qué pasa sin ella?

Preguntas que podrían aparecer en el parcial (I)

1. Definí *vecindad* y *óptimo local*. ¿Por qué la búsqueda local termina? ¿Qué garantiza?
2. ¿Qué diferencia hay entre una heurística y un algoritmo de aproximación? Da un ejemplo de cada uno para el mismo problema.
3. Enunciá el algoritmo de matching maximal para vertex cover y demostrá que es una 2-aproximación. ¿Es ajustada la cota?
4. Enunciá el algoritmo del árbol generador mínimo para TSP métrico y demostrá su factor. ¿Dónde se usa la desigualdad triangular? ¿Qué pasa sin ella?

Respuestas modelo, de tres a cinco líneas cada una, en el documento del pizarrón.

Preguntas que podrían aparecer en el parcial (II)

5. ¿Qué es una metaheurística? Describí las dos fases de GRASP y el rol de α. ¿Qué pasa con $\alpha = 0$? ¿Y con $\alpha = 1$?

Preguntas que podrían aparecer en el parcial (II)

5. ¿Qué es una metaheurística? Describí las dos fases de GRASP y el rol de α. ¿Qué pasa con $\alpha = 0$? ¿Y con $\alpha = 1$?
6. ¿Qué es una heurística admisible? ¿Y consistente? ¿Qué garantiza A* con h consistente, y por qué? ¿Qué algoritmo es A* con $h \equiv 0$?

Preguntas que podrían aparecer en el parcial (II)

5. ¿Qué es una metaheurística? Describí las dos fases de GRASP y el rol de α. ¿Qué pasa con $\alpha = 0$? ¿Y con $\alpha = 1$?
6. ¿Qué es una heurística admisible? ¿Y consistente? ¿Qué garantiza A* con h consistente, y por qué? ¿Qué algoritmo es A* con $h \equiv 0$?
7. ¿Es 2-opt un algoritmo de aproximación para el TSP? ¿Es el greedy por grado una 2-aproximación para vertex cover? Justificá.

Preguntas que podrían aparecer en el parcial (II)

5. ¿Qué es una metaheurística? Describí las dos fases de GRASP y el rol de α. ¿Qué pasa con $\alpha = 0$? ¿Y con $\alpha = 1$?
6. ¿Qué es una heurística admisible? ¿Y consistente? ¿Qué garantiza A* con h consistente, y por qué? ¿Qué algoritmo es A* con $h \equiv 0$?
7. ¿Es 2-opt un algoritmo de aproximación para el TSP? ¿Es el greedy por grado una 2-aproximación para vertex cover? Justificá.
8. Se tiene una heurística h admisible pero no consistente. ¿Puede A* (sin reabrir vértices) devolver un camino no óptimo? Da un ejemplo o demostrá que no.

Tips

1. Antes de decir «heurística», «aproximación» o «metaheurística», preguntarse **qué se cede** (optimalidad, tiempo) y **qué se garantiza**. La respuesta es la definición.

Tips

1. Antes de decir «heurística», «aproximación» o «metaheurística», preguntarse **qué se cede** (optimalidad, tiempo) y **qué se garantiza**. La respuesta es la definición.
2. Toda garantía de aproximación se demuestra contra una **cota** de OPT que sí se puede razonar. Encontrar la cota es encontrar la demostración.

Tips

1. Antes de decir «heurística», «aproximación» o «metaheurística», preguntarse **qué se cede** (optimalidad, tiempo) y **qué se garantiza**. La respuesta es la definición.
2. Toda garantía de aproximación se demuestra contra una **cota** de OPT que sí se puede razonar. Encontrar la cota es encontrar la demostración.
3. Toda hipótesis que se usa (desigualdad triangular, pesos no negativos, consistencia) hay que **nombrarla**; sin ella el teorema suele ser falso, y conocer el contraejemplo es parte de entenderlo.

Tips

1. Antes de decir «heurística», «aproximación» o «metaheurística», preguntarse **qué se cede** (optimalidad, tiempo) y **qué se garantiza**. La respuesta es la definición.
2. Toda garantía de aproximación se demuestra contra una **cota** de OPT que sí se puede razonar. Encontrar la cota es encontrar la demostración.
3. Toda hipótesis que se usa (desigualdad triangular, pesos no negativos, consistencia) hay que **nombrarla**; sin ella el teorema suele ser falso, y conocer el contraejemplo es parte de entenderlo.
4. En búsqueda local, el diseño es la **vecindad** y la **evaluación incremental**. Lo demás es un `while`.

Tips

1. Antes de decir «heurística», «aproximación» o «metaheurística», preguntarse **qué se cede** (optimalidad, tiempo) y **qué se garantiza**. La respuesta es la definición.
2. Toda garantía de aproximación se demuestra contra una **cota** de OPT que sí se puede razonar. Encontrar la cota es encontrar la demostración.
3. Toda hipótesis que se usa (desigualdad triangular, pesos no negativos, consistencia) hay que **nombrarla**; sin ella el teorema suele ser falso, y conocer el contraejemplo es parte de entenderlo.
4. En búsqueda local, el diseño es la **vecindad** y la **evaluación incremental**. Lo demás es un `while`.
5. Un resultado experimental sin instancias, semillas, referencia y repeticiones no es un resultado. Es una anécdota.

Tips

1. Antes de decir «heurística», «aproximación» o «metaheurística», preguntarse **qué se cede** (optimalidad, tiempo) y **qué se garantiza**. La respuesta es la definición.
2. Toda garantía de aproximación se demuestra contra una **cota** de OPT que sí se puede razonar. Encontrar la cota es encontrar la demostración.
3. Toda hipótesis que se usa (desigualdad triangular, pesos no negativos, consistencia) hay que **nombrarla**; sin ella el teorema suele ser falso, y conocer el contraejemplo es parte de entenderlo.
4. En búsqueda local, el diseño es la **vecindad** y la **evaluación incremental**. Lo demás es un `while`.
5. Un resultado experimental sin instancias, semillas, referencia y repeticiones no es un resultado. Es una anécdota.
6. Verifiquen la consistencia de h con una desigualdad, no con una intuición. Y $h \equiv 0$ siempre sirve: A* nunca es peor que Dijkstra, sólo paga por calcular h.

Bibliografía

- J. Kleinberg, É. Tardos, *Algorithm Design*: cap. 11 (aproximación), 12.1–12.4 (búsqueda local). La referencia principal.
- T. Cormen et al., *Introduction to Algorithms*: cap. 35 (aproximación: 35.1 vertex cover, 35.2 TSP).
- S. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach*: cap. 3 (A*, admisibilidad, consistencia), cap. 4 (búsqueda local, simulated annealing), 6.4 (min-conflicts).
- M. Resende, C. Ribeiro, *Optimization by GRASP* (2016); T. Feo, M. Resende, «Greedy randomized adaptive search procedures», *J. Global Optimization* 6 (1995).
- D. Johnson, L. McGeoch, «The traveling salesman problem: a case study in local optimization» (1997).
- D. Williamson, D. Shmoys, *The Design of Approximation Algorithms* (2011, libre en `designofapproxalgs.com`).

¿Preguntas?